



Bachelorarbeit

Entwurf und Implementierung eines modularen Multisensorknotens für Smart Home-Anwendungen

Design and implementation of a modular multi-sensor node for smart home applications

Verfasser: Dennis Makein

Matrikelnummer: 420832

Abgabedatum: 31. Juli 2025

Entwurf Mikroelektronischer Systeme (EMS)

Prüfer: Prof.Dr.-Ing. Norbert Wehn

Betreuer: Dr.Ing. Christian De Schryver

Eidesstattliche Erklärung

Ich versichere hiermit an Eides statt, die vorliegende Arbeit gemäß BPO/MPO Elektrotechnik und Informationstechnik bis auf die durch meinen Betreuer gewährte Unterstützung ohne Hilfe Dritter selbstständig angefertigt, alle benutzten Quellen und Hilfsmittel einschließlich des Internets vollständig und genau angegeben und alles kenntlich gemacht zu haben, was aus Arbeiten anderer unverändert, mit Abkürzung oder sinngemäß übernommen wurde.

Kaiserslautern, den 31. Juli 2025

Dennis Makein

Kurzfassung

Diese Arbeit beschäftigt sich mit der Anfertigung eines Multisensors für die Benutzung in einem Smart Home. Dabei wird Matter als Verbindungsstandard gewählt. Gerade, da dieser viele vorhandener Standards vereinigt, ist eine Erweiterung der bereits vorhandenen open-source Software von Interesse. Der Multisensor kann die Temperatur, Luftfeuchtigkeit, Luftqualität und eine Präsenzmeldung anzeigen. Dieser wurde zunächst auf einem Breadboard aufgebaut, auf dessen Grundlage eine PCB erstellt wurde. Mithilfe von HomeAssistant können die gemessenen Werte angezeigt und folgend weiter als Einbindung in Automationen verwendet werden.

Abstract

This thesis deals with the creation of a multisensor for use in a smart home system. Matter is chosen as the connection standard. Since it combines many existing standards, an extension of the existing open-source software is particularly interesting. The multisensor can display temperature, humidity, air quality, and a presence detector. It was initially built on a breadboard, from which a PCB was created. Using HomeAssistant, the measured values can be displayed and subsequently integrated into automation systems.

Inhaltsverzeichnis

Eidesstattliche Erklärung	II
Kurzfassung Abstract	III
1 Einleitung	1
2 State of the Art	2
2.1 Mikrocontroller	2
2.2 SmartHome Standards	3
2.2.1 ZigBee	3
2.2.2 Z-Wave	4
2.2.3 BLE	4
2.2.4 Wi-Fi	4
2.3 SmartHome Produkte	5
2.3.1 Homematic IP	5
2.3.2 Shelly	5
2.3.3 Bosch Smart Home	6
2.3.4 Eve Home	6
2.3.5 Aqara	6
2.3.6 Übersicht	7
3 Theoretische Grundlagen	8
3.1 Sensorenauswahl	8
3.2 Grundlagen von I2C	9
3.3 Sensorenbeschreibung	10
3.3.1 IST HYT 271	10
3.3.2 Joy-it SEN-CCS811V1	12
3.3.3 Omron D6T1A01	13
3.4 MQTT	15
3.5 Matter	16
3.5.1 Weitere Begriffe	18
3.5.2 Thread-Protokoll	18

4 Durchführung	20
4.1 Aufbau auf dem Breadboard	20
4.2 Einrichtung VS Code mit der ESP-IDF	21
4.3 Matter Sensors	21
4.3.1 Aufbau	21
4.3.2 Drivers	22
4.3.3 app_main	23
4.4 PCB	24
4.5 SmartHome Server	26
4.6 Anwendung	27
5 Ergebnisse und Auswertung	29
6 Schlussfolgerung und Ausblick	30
Literaturverzeichnis	38
Abkürzungsverzeichnis	39
Glossar	40
Tabellenverzeichnis	41
Abbildungsverzeichnis	42
Anhang	43
A.1 WSL: flash + monitor	43

1 Einleitung

Das Thema dieser Arbeit ist der Entwurf eines Multisensorknotens für Smart Home-Anwendungen. Hierbei wird der Matter-Standard verwendet. Der große Vorteil von Matter ist das Verbinden mehrerer Smart Home-fähiger Geräte, die nicht vom selben Hersteller sind. Viele Unternehmen, wie z.B. Amazon, Google, Apple, Samsung, Bosch, uvm., unterstützen bereits den Matter Standard in ihren Produkten.

Durch Matter besteht die Möglichkeit für den Endverbraucher, unabhängig vom vertreibenden Unternehmen, ein Produkt zu wählen, welches auf die Bedürfnisse des Nutzers zugeschnitten ist. Das einzige Problem an Matter ist, dass es noch nicht sehr lange veröffentlicht ist und somit die Anzahl an möglichen Einsätzen, gerade in spezielleren Produkten, anhand der noch nicht vorhandenen Software scheitert. Da diese aber open-source, d.h. für jeden nutzbar und erweiterbar, ist, werden immer mehr Lösungen geboten um neue Produkte und Funktionen einzubinden. Auch deshalb wird bei diesem Projekt Matter gewählt, um die vorhandene Software zu erweitern oder zu verbessern. Hilfreich dabei ist, dass viele Mikrocontroller, wie der hier verwendete ESP32C6, schon eine Kompatibilität mit Matter vorweisen.

Die Attribute des Multisensors sind: Temperatur, Luftfeuchtigkeit, Luftqualität und eine Präsenzmessung. Diese werden als ein Gerät im Verlauf der Arbeit auf einem Smart Home-Server dargestellt. Diese Werte können dann als IST-Werte mit den SOLL-Werten verglichen und infolge dessen manuell oder auch automatisch mit anderen Smart Home-Geräten geregelt werden.

Des Weiteren wird für den Multisensor eine PCB (printed circuit board) entworfen. Eine Platine ist im Vergleich zu einem Aufbau auf einem Breadboard mit Jumper Wires professioneller und näher an einem fertigen Produkt, wie es auf dem Markt vertrieben werden kann. Durch den Schaltplan ist diese auch einfach nachvollziehbar, wobei der Aufbau auf dem Breadboard hingegen bei größeren Schaltungen unübersichtlicher werden kann. Dieser eignet sich vor allem zu Beginn für Testzwecke. Folgend werden zuerst die Grundlagen und der State of the Art von Smart Home Systemen und -Standards und möglichen Mikrocontrollern dargestellt. Anschließend wird auf die Hardware und den, für das Projekt wichtigen, Matter-Standard eingegangen. Als Nächstes wird die Durchführung der praktischen Arbeit beleuchtet, wobei z.B. der Code oder auch der Einsatz des Sensors aufgezeigt wird. Abschließend erfolgt eine Reflexion der Arbeit gefolgt von dem Fazit.

2 State of the Art

2.1 Mikrocontroller

Im Folgenden wird ein kurzer Überblick über verschiedene Mikrocontroller geschaffen. Darunter auch der in diesem Projekt verwendete ESP32-C6:

Der **ESP32-C6** ist ein Mikrocontroller der Firma Espressif, welcher in vielen Bereichen seinen Einsatz finden kann. Gerade in neuartigen IoT-basierten Anwendungen kann der ESP32-C6 aufgrund von seiner Vielseitigkeit überzeugen. Der Controller unterstützt Wi-Fi 6(2.2.4), Bluetooth 5 (/ BLE(2.2.3)), ZigBee(2.2.1) und auch Thread(3.5.1). Anwendungsbeispiele mit dem ESP können z.B. im SmartHome-Bereich, der Gesundheitsüberwachung, einem POS-System (Point of Sale) oder auch generellen Einsatz in der Industrie gefunden werden. Die Software des ESP ist open-source, was für die Entwicklung und Innovation einen großen Vorteil bietet, da diese durch die Zusammenarbeit Vieler schneller vorangetrieben werden können. Die Hardware des Chips hingegen ist nicht open-source und genauere Details sind unter Verschluss gehalten. Bekannt ist z.B., dass der Hauptbaustein dieses Chips ein 32 Bit RISC-V Prozessor ist. Dieser ist im ESP32-C6-WROOM verbaut. Der WROOM kann Teil eines DevKits sein, mit welchem auch in diesem Projekt gearbeitet wird. [1]

Als weiterer großer Vertreiber im Markt für Mikrocontroller ist vor allem noch **Arduino** zu nennen. Arduino ist bekannt für die große Anzahl an möglichen Boards, wie z.B. die bekannten NANO oder UNO. Viele der Arduino-Boards sind speziell für die IoT-Benutzung ausgelegt. Sie sind mit BLE(2.2.3) und Wi-Fi(2.2.4) ausgestattet, was einen vielseitigen Einsatz z.B. im Heimnetzwerk, zur Automatisierung oder auch mit Robotern ermöglicht. Hier ist ebenfalls die Software, wie beim ESP, open-source. Als Chips der Arduino-Boards werden vermehrt Atmel AVR Mikrocontroller (z.B. die ATmega-Reihe) verwendet. Zudem gibt es auch Erweiterungen für die Boards, Shields genannt, die noch einmal speziellere Funktionen erfüllen können. Des Weiteren ist die eigene integrierte IDE (Entwicklungsumgebung) zu erwähnen, in welcher die Programmierung in einer einfacher gehaltenen Version von C++ erfolgt. [2, 3, 4]

Ein anderer bekannter Hersteller ist **Raspberry Pi**. Hier gibt es ebenfalls viele verschiedene vorgefertigte Boards, Module oder auch Erweiterungen, aus denen anhand der speziellen Anforderungen ausgewählt werden kann. Diese werden ebenso wie der ESP und Arduino in vielen Gebieten, wie z.B. IoT, SmartHome oder in der Industrie, benutzt. Dabei können die Schnittstellen zwischen den Boards variieren, aber die meisten sind Bluetooth(2.2.3), Wi-Fi(2.2.4), HDMI

und USB-fähig. Die Schaltpläne der Boards sind teilweise dokumentiert und viele Zusatzplatinen sind auch open-source. Die Prozessoren hingegen sind proprietär. Das Betriebssystem des Raspberry Pi ist hingegen vollständig open-source. Es ist eine auf Debian basierende Linux Distribution, welche viele Programme wie z.B. Python, HomeAssistant oder Docker unterstützt. [5]

2.2 SmartHome Standards

Zwar werden hier in diesem Projekt nur Matter und die Charakteristiken von MQTT benutzt, jedoch werden im Folgenden noch diverse andere Standards für Multi-Sensor Applikationen in Smart-Home Netzwerken aufgezeigt, um einen allgemeinen Überblick über das Thema zu verschaffen.

2.2.1 ZigBee

ZigBee ist mit dem IEEE 802.15.4 Standard eingeführt worden. Zu ZigBee gehören Unternehmen wie Invensys, Honeywell, Mitsubishi Electric, Motorola, Philips und viele mehr. ZigBee ist sehr stark für verbindungslose Sensoren ausgelegt. Der Standard ist kosteneffizient, stromsparend und hat eine geringe Latenz. Dabei ist die Datenrate im Vergleich zu Bluetooth oder Wi-Fi um ein Vielfaches niedriger, wohingegen bei ZigBee die mögliche Anzahl an Nodes pro Netzwerk positiv herausragt (653356 Nodes). Die Distanz zwischen zwei Geräten kann bis zu 50 Meter betragen. Dabei kann jedes Gerät die Daten an ein anderes Gerät weiterleiten. Es gibt zwei verschiedenen Arten von Geräten in dem Netzwerk: fully functional device (FFD) und reduced functional device (RFD). Ein FFD kann mit jedem Gerät kommunizieren, neben der Endgerätsfunktion auch als Netzwerk Koordinator (network coordinator) oder auch Router dienen und in jeder Topologieart eingesetzt werden. Ein RFD hingegen kann nur mit einem Netzwerk Koordinator kommunizieren und nur in einer Stern-Topologie vernetzt sein. Ein RFD kann z.B. ein klassischer Sensor sein, welcher sich auch, typisch für ZigBee, in einen Schlaf-Modus (snoot) versetzen kann, wenn dieser keine Anfrage erhält, um somit Energie zu sparen. Außerdem ist im Rahmen des Sicherheitsaspektes zu erwähnen, dass Geräte in einem ZigBee-Netzwerk eine Liste an vertrauenswürdigen Geräten innerhalb des Netzwerks besitzen. Hier wird ein kryptischer Schlüssel benutzt, welcher den Geräten bei der Aufnahme in das Netzwerk mitgegeben wird (ähnlich wie bei Matter(3.5)). [6, 7]

2.2.2 Z-Wave

Z-Wave ist ein Funkstandard mit Niedrigenergie-Funkwellen. Das System benutzt im Vergleich zu den anderen aufgelisteten Protokollen eine Frequenz von 868,42 MHz, sodass keine Interferenz mit diesen entstehen. Die Manchester-Kodierung, die bei diesem Standard verwendet wird, ist eine Form der Phasenmodulation. Ein Z-Wave Netzwerk ist in der Mesh-Netzwerktopologie aufgebaut. Jedes Netzwerk besitzt seine eigene Netzwerk-ID. Dabei kann jedes Netzwerk bis zu 232 Knotenpunkte besitzen. Nur Knoten mit gleicher Netzwerk-ID können miteinander kommunizieren. Zur Einrichtung und Administrierung eines Netzwerkes wird ein zentraler Controller benötigt. Je mehr Geräte in einem Netzwerk vorhanden sind, desto stärker und weiter reicht das gesendete Signal. Zudem ist eine Erweiterung der Reichweite durch einen Repeater möglich. Z-Wave besitzt eine Low-Power-RF-Kommunikationstechnologie für Mesh-Netzwerke ohne Koordinatorknoten. Für die Sicherheit in den Netzwerken sorgt das AES-128 Verschlüsselungssystem. Negativ zu notieren ist, dass Z-Wave nur mit einer geringen Datenübertragungsgeschwindigkeit von 100 Kbit/s arbeitet, da das System voll und ganz auf den Gebrauch im Smart-Home Bereich ausgelegt ist. [8]

2.2.3 BLE

Bluetooth ist ein verbindungsloser Standard, der im alltäglichen Leben Gebrauch findet. BLE (Bluetooth low energy) wurde in der Version Bluetooth 4 mit eingeführt. Wie LE schon sagt, war es das Ziel Bluetooth energieeffizienter zu gestalten. Dies wurde vor allem durch die immer größere und auch ständige bzw. länger andauernde Vernetzung von Geräten, z.B. IoT (Internet of Things), miteinander ein wichtiger Vorteil. Ein Beispiel hierzu ist der Einsatz von BLE zur Geräteortung. Dabei arbeitet BLE, genauso wie das klassische Bluetooth, im 2,4 GHz Frequenzband. Ein Gerät kann neben Punkt-zu-Punkt-, wie beim herkömmlichen Bluetooth, auch in Broadcast- oder Mesh-Topologie angebunden sein. [9, 10]

2.2.4 Wi-Fi

Wi-Fi (Wireless Fidelity) ist neben Bluetooth die wohl am weitesten bekannte und verbreitete Technologie. Bei Wi-Fi gibt es zwei verschiedene Frequenzbänder, das 2,4 GHz und das 5 GHz Band. Wi-Fi basiert auf der IEEE 802.11 und der aktuell neuste Standard ist Wi-Fi 7. WPA3 (Wi-Fi Protected Access 3) ist für die Sicherheit der Verbindung zuständig. Dabei gibt es wiederum zwei Arten, WPA3-Personal, welches sich nach dem klassischen Verbraucher richtet, und WPA3-

Enterprise, welches auf die Sicherheit für Unternehmen spezialisiert ist. Wi-Fi ist der Standard, der mit Abstand am meisten Stromleistung benötigt. Dafür kann es einen hohen Datenverkehr bei niedriger Latenz handeln und die Verbindung auch bei einer größeren Reichweite aufrecht erhalten. [11, 12]

2.3 SmartHome Produkte

In diesem Kapitel soll ein Überblick für die am Markt vorhandenen Produkte geschaffen werden. Hierzu ist eine Auswahl von fünf Systemen getroffen worden.

2.3.1 Homematic IP

Homematic IP wurde von der Firma eQ-3 entwickelt. Bei Homematic IP ist eine Funk- und kabelgebundene Kommunikation möglich. Diese beiden Möglichkeiten können auch miteinander kombiniert werden (Advanced Routing). Dabei gibt es schon über 150 eigene kabelgebundene, sowie Funk-Produkte, aus denen eine Auswahl getroffen werden kann. Das verwendete Funkprotokoll ist resistent gegenüber Ausfall des Internets und benutzt weiterhin eine Frequenz von 868 MHz, wodurch Störungen durch andere Funksysteme vermieden werden. Des Weiteren gibt es eine eigene App, mit welcher die Einrichtung oder die Steuerung der Geräte vorgenommen werden kann. Außerdem ist die Steuerung auch über Sprachassistenten wie z.B. Alexa oder Google Assistant möglich. Bei dem Thema Sicherheit kann Homematic IP ebenfalls relevante Daten vorweisen: Homematic IP ist VDE- und AV-Test-geprüft, der Verschlüsselungsstandard AES-128 und das Protokoll Bidirectional Communication Standard (BidCoS) kommen zum Einsatz und nur berechtigte Sender können Befehle an Geräte senden (Challenge-Response-Authentifizierungsverfahren). [13, 14]

2.3.2 Shelly

Hinter Shelly steckt die Firma Allterco Robotics. Hier erfolgt die Kommunikation über WLAN / LAN, Bluetooth, KNX oder auch Z-Wave. Das bedeutet, dass bei Shelly eine lokale Steuerung ohne Cloud möglich ist. Shelly bietet neben der fast schon standardmäßigen App zum Einrichten und Steuern des Smart Homes viele weitere Funktionen, wie z.B. ein Feature zur Messung des Stromverbrauchs. Die Sicherheit der Kommunikation wird durch den neusten Wi-Fi-Standard und WPA3 garantiert. Shelly bietet eine hohe Kompatibilität bei Assistenten (Amazon Alexa, Google Home, Smart Things), Servern (HomeAssistant, Homey, OpenHab) und Proto-

kollen (MQTT, Matter, KNXNet, uvm.). [15, 16, 17, 18]

2.3.3 Bosch Smart Home

Beim Smart Home System der Firma Bosch wird auf eine Kommunikation über Funk gesetzt. Wie bei den anderen Systemen kann eine eigene App zur Einrichtung und Steuerung des Systems verwendet werden. Viele Produkte von Bosch Smart Home sind kompatibel mit Matter. Zudem ist ein flexibler Wechsel zwischen Bosch Smart Home und Matter möglich. Außerdem besitzt das System generell eine hohe Kompatibilität mit anderen Systemen. Dabei sind unter anderem Unternehmen wie Amazon, Apple, Google (Sprachassistenten), Phillips, Buderus zu nennen. Das AES128- und das TLS(Transport Layer Security)-Verschlüsselungsverfahren sind die Hauptfaktoren für eine sehr hohe Sicherheit des Systems.

[19, 20, 21]

2.3.4 Eve Home

Eve, früher Elgato, ist ein Teil von ABB und hat sich auf die Entwicklung von Geräten für den Apple Standard spezialisiert. Geräte von Eve Home können über WLAN mit AppleHome, Thread oder auch Bluetooth kommunizieren und sind vollständig in HomeKit integriert. Durch die Unterstützung von Matter können die Geräte auch plattformübergreifend sicher kommunizieren. Die Kommunikation erfolgt lokal, das heißt dass keine Abhängigkeit von einer Cloud besteht. Somit werden keine Daten gespeichert, was vor allem zum Datenschutz des Benutzers beiträgt. Auch hier gibt es eine App, mit welcher die Steuerung, Verwaltung und Erweiterung des System vorgenommen werden kann. Durch die Verwendung von HomeKit wird die Sicherheit der Kommunikation gewährleistet. Hier wird ebenfalls das AES-128- und das TLS-Verschlüsselungsverfahren verwendet.

[22, 23, 24, 25, 26, 27]

2.3.5 Aqara

Aqara ist die Marke für Smart Home Anwendungen der Firma Lumi. Die Kommunikation und die generelle Sicherheit der Verbindung läuft über den ZigBee-Standard (AES-128). Einige, vor allem neuere, Geräte verwenden auch mit Matter oder Thread. Das System, ist ebenso wie die anderen Systeme, mit vielen großen Playern am Markt kompatibel. Alexa, GoogleHome, SamsungSmartThings sind nur einige Beispiele. Ebenfalls gibt es eine App mit der die Steuerung oder das Auslesen der Geräte erfolgen kann.

[28, 29, 30]

2.3.6 Übersicht

Im folgenden sind die wichtigsten Eigenschaften der Systeme noch einmal zusammengefasst dargestellt:

Tab. 2.1: Smart-Home Systems

	Homematic IP	Shelly	Bosch Smart Home	Eve Home	Aquara
Kommunikation	Funk und Kabel	WLAN/LAN/ Bluetooth/ KNX/Z-Wave	Funk (WLAN)	WLAN Thread Bluetooth	ZigBee Thread Matter
Sicherheit	AES-128	Wi-Fi-Sicherheit und WPA3	AES-128 und TLS	AES-128 und TLS	ZigBee- Standard (AES-128)
Kompatibilität	Alexa oder Google Assistant uvm.	sehr hoch	Matter, Wechsel zw. Matter und Bosch Smart Home mögl. viele weitere	AppleHome und Matter	Alexa, GoogleHome, Samsung- SmartThings, uvm.

Alle Systeme besitzen eine App, mit welcher diese eingerichtet und gesteuert werden können.

3 Theoretische Grundlagen

3.1 Sensorenauswahl

Zu Beginn der Arbeit wurde anhand der Aufgabenstellung eine Vorauswahl für die verschiedenen, zu verwendenden Sensortypen des Multisensors herausgesucht. Diese sind ein Temperatur-, ein Feuchtigkeits-, ein Luftqualitätssensor, sowie ein Standardpräsenzmelder.

Tab. 3.1: Multisensor (Temperatur- und Feuchtigkeitssensor)

Bezeichnung	Interface	Betriebsspannung	Messbereich	Genauigkeit	Kosten
IST Sensor Feuchte- und Temperatur-Sensor HYT 271 [31]	I2C	2,7V bis 5,5V	0 bis 100%rF, -40 bis 125°C	±1,8%rF, ±0,2°C	36,99€
Amphenol Digital Temperatur- und Feuchtigkeits- sensor CC2D23-SIP [32]	I2C	2,3V bis 5,5V	0 bis 100%RH, -40 bis 125°C	±2%rF, ±0,3°C	13,80€

Tab. 3.2: Luftqualitätssensor

Bezeichnung	Interface	Betriebsspannung	Messung	Kosten
Joy-it SEN-CCS811V1 Sensor-Modul [33]	I2C	3V bis 5V	0 - 1187 Teile/Milliarde (TVOCs) 400 - 8192 Teile/Million (eCO2)	36,99€

Tab. 3.3: Standardpräsenzmelder

Bezeichnung	Interface	Betriebsspannung	Messbereich	Kosten
Omron D6T1A01 Thermischer MEMS-Präsenzsensoren [34]	I2C	4,5V bis 5,5V (3,5mA)	Kreisdurchmesser: 222cm (2m Höhe)	13,95€
Omron D6T-44L-06H Wärmesensor IC-Näherungssensoren [35]	I2C	4,5V bis 5,5V (5mA)	162cm x 169cm (2m Höhe)	49,78€

Anschließend wurden die Sensoren IST HYT 271, Joy-it SEN-CCS811V1 und Omron D6T1A01 durch den Betreuer ausgewählt. Diese Komponenten werden im Anschluss zum I2C-Kapitel noch einmal genauer dargestellt werden.

3.2 Grundlagen von I2C

Das I2C-Interface ist ein Bussystem, das auf der grundlegenden Master-Slave-Beziehung funktioniert. D.h. dass ein Master (z.B. ein Mikrocontroller) und viele Slaves (z.B. Sensoren) existieren. Hierbei bestimmt der Master die Art der Datenübertragung, ob dieser Daten an die Slaves sendet oder diese von ihnen empfängt. Der I2C-Bus ist ein synchroner serieller Zweidrahtbus, was für die Datenübertragung bedeutet, dass es eine Daten- und eine Taktleitung gibt (SDA und SDL). Zusätzlich wird noch eine GND-Leitung benötigt.

Die Datenübertragung erfolgt über ein I2C-Protokoll, welchem alle I2C-fähigen Komponenten unterliegen. Der grundlegende Aufbau des Protokolls lässt sich in 7 Teile gliedern:

Bei der Startbedingung wird ein Aufwecksignal vom Master an alle Slaves geschickt, wodurch die Slaves auf die darauf folgenden weiteren Teile aufmerksam gemacht werden.

Anschließend folgt die 7-Bit Adresse des Slaves, mit welchem der Master die Datenübertragung durchführen möchte.

Als nächstes folgt ein Read- oder Write-Bit, mit welchem angegeben wird, ob der Slave vom Master Informationen empfängt (liest) oder ihm Informationen bereitstellt (schreibt).

Auf dieses folgt ein anschließendes ACK-Bit des Slaves.

Aufgrund der R/W-Unterscheidung entscheidet sich auch im Folgenden welcher von beiden die Daten sendet. Diese Daten werden in Paketen von 8 Bit, also einem Byte, versendet.

Jedes gesendete Datenpaket wird mit einem ACK-Bit vom Gegenspieler bestätigt, es sei denn dieser möchte die Verbindung beenden. Hier wird dann ein negiertes ACK-Bit gesendet.

Abschließend beendet der Master die Datenübertragung mit einer Stoppbedingung. [36, 37]

3.3 Sensorenbeschreibung

3.3.1 IST HYT 271



Abb. 3.1: IST HYT 271

Der HYT 271 der IST-Familie ist ein Temperatur- und Feuchtigkeitssensor. Diese generieren anhand des polymerbasierten Sensorelements Daten, welche in relative Feuchtigkeits- und Temperaturwerte umgerechnet werden können.

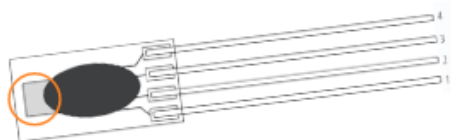


Abb. 3.2: aktiver Sensor des HYT271

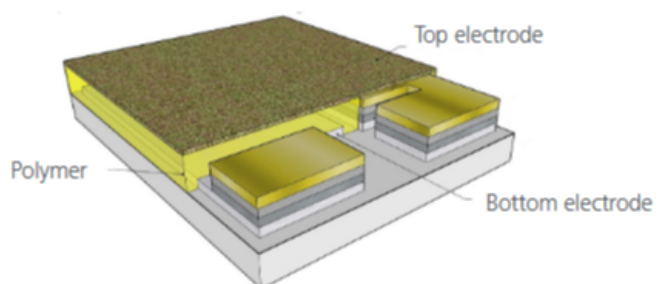


Abb. 3.3: Aufbau des Sensors

Dieser Sensor beruht auf dem beschriebenen Protokoll im I2C-Kapitel 3.2. Der Sensor hat zwei verschiedene Kommandos: Data Fetch (DF) (Abb.3.5) und Measurement Request (MR) (Abb.3.4). Bei beiden Kommandos ist der Anfang des Protokolls gleich: Zuerst wird das Start-Bit übermittelt, gefolgt von der sieben Bit Geräteadresse. Danach folgt das Write/Read-Bit, welches beim Measurement Request als Write und beim Data Fetch als Read ausgeführt wird. Nach dem Write/Read-Bit folgt ein ACK-Bit des Sensors. Beim MR schließt das Stopp-Bit das Protokoll ab, wohingegen beim DF jetzt erst die Daten vom Sensor übermittelt werden. Dabei werden in vier Byte-Blöcken die Luftfeuchtigkeitswerte (erstes und zweites Byte) und die

Temperaturwerte (drittes und viertes Byte) gesendet. Nach den ersten drei Bytes folgt jeweils ein ACK-Bit des Masters. Nach dem vierten Byte folgt ein NACK-Bit mit dem anschließenden Stopp-Bit.



Abb. 3.4: HYT271 Measurement Request Protokoll

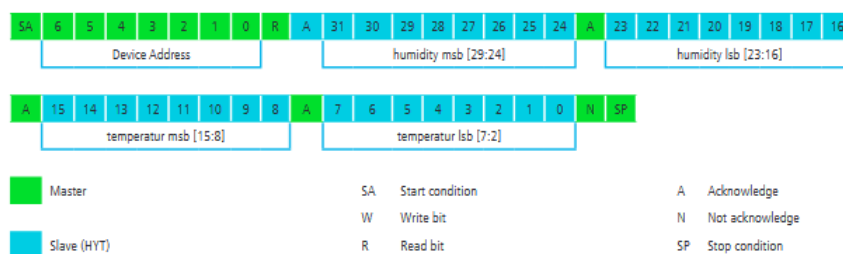


Abb. 3.5: HYT271 Data Fetch Protokoll

Um nun aus diesen Bytes reale Datenwerte generieren zu können, müssen zuerst die Luftfeuchtigkeitsblöcke, eins und zwei, sowie die Temperaturblöcke, drei und vier, zusammengeführt werden. Bei den 16 Bits der Luftfeuchtigkeit muss man nun die ersten zwei Bits, die Bits die am weitesten links liegen, zu Nullen maskieren und diese 14 Bit dann in eine Dezimalzahl umwandeln. Abschließend wird diese Zahl mit 100 multipliziert und durch 16383 geteilt. Somit erhält man den Luftfeuchtigkeitswert in der Einheit %RH. Dabei kann die Luftfeuchtigkeit die Werte 0-100%RH annehmen. Der dezimale Wert 0 entspricht auch 0%RH und der dezimale Wert 16383 (hexadezimal: 3FFF) entspricht 100%RH.

Um den Temperaturwert aus den Byteblöcken drei und vier zu erhalten, muss man diese ebenfalls von 16 Bits auf 14 Bits verkleinern. Hier werden die letzten zwei Bits, die Bits die am weitesten rechts liegen, gestrichen. Die aus den 14 Bit entstandene Dezimalzahl muss noch mit dem Faktor 165/16383 multipliziert werden und abschließend wird hier noch der Wert 40 abgezogen. Somit erhält man den Temperaturwert in °C. Dieser kann die Werte im Bereich von -40°C bis zu 125°C annehmen. Der dezimale Wert 0 entspricht -40°C und der dezimale Wert 16383 (hexadezimal: 3FFF) entspricht 125°C.

Diese beiden Umrechnungen sind in Abb. 3.6 anhand von Beispielwerten dargestellt:

Example:

	Byte 1	Byte 2	Byte 3	Byte 4
	31 dec	109 dec	96 dec	72 dec
bin	0001.1111	0110.1101	0110.0000	0100.1000
	Humidity 14 bit right-adjusted		Temperature 14 bit left-adjusted	
hex	1F6D		1812	
dec	$8045 \times 100/16383 =$		$6162 \times 165/16383 - 40 =$	
	49.1 %RH		22.06 °C	

Abb. 3.6: HYT271 Umrechnung in reale Werte

[38]

3.3.2 Joy-it SEN-CCS811V1

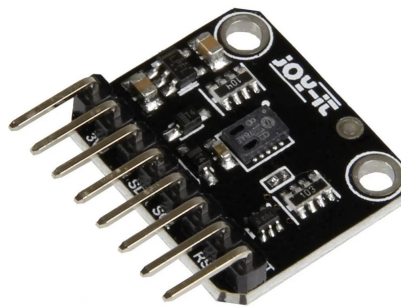


Abb. 3.7: Joy-it SEN-CCS811V1

Der Joy-it SEN-CCS811V1 ist ein Gassensor, mit welchem die Qualität der Luft gemessen werden kann. Dabei kann man die flüchtigen organischen Verbindungen (VOC) messen, welche man als TVOC oder eCO₂ ausgeben lassen kann. Dieser Sensor beruht ebenfalls auf dem I2C-Protokoll, welches im vorherigen Kapitel beschrieben wurde.

Hier werden ebenfalls eine Write- und anschließend eine Read-Operation, wie beim HYT271, durchgeführt. Der Aufbau der Operationen ist zu Beginn wieder gleich (Abb. 3.8): Sie starten mit einem Start-Bit, gefolgt von der Geräteadresse. Anschließend folgt wieder das Write/Read-Bit, welches dazu führt, dass die beiden Operationen im folgenden, nach dem folgenden ACK-Bit des Slaves, unterschiedlich ablaufen. Bei der Write Operation folgt die Registeradresse, auf welche wieder ein ACK des Slaves und abschließend noch das Stopp-Bit folgt. Die Read Operation ließt nach dem ACK-Bit die Daten von der gegebenen Registeradresse. Abschließend folgt ein NACK des Masters und das Stopp-Bit. Mit der Registeradresse wird festgelegt, aus welchem

Register die Daten der Read Operation gelesen werden. Um die TVOC und eCO₂ Daten auszulesen müssen die Daten aus dem ALG_RESULT_DATA Register (Abb. 3.9) mit der Adresse 0x02 ausgelesen werden. Das Register besteht aus acht Bytes, wobei Byte null und eins den eCO₂-Wert und Byte zwei und drei den TVOC-Wert beinhalten.

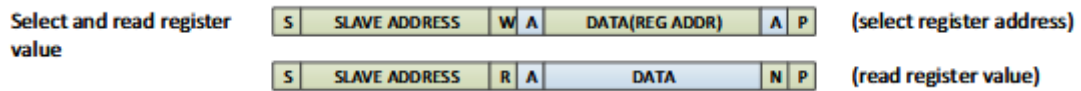


Abb. 3.8: CCS811 Protokoll

Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6 & 7
eCO ₂ High Byte	eCO ₂ Low Byte	TVOC High Byte	TVOC Low Byte	STATUS	ERROR_ID	See RAW_DATA

Abb. 3.9: CCS811 Aufbau des Algorithm Results Registers

Der eCO₂-Wert kann Werte im Bereich von 400 ppm bis 8192 ppm annehmen. Der TVOC-Wert hingegen kann Werte im Bereich von 0 ppb bis zu 1187 ppb annehmen.

[39]

3.3.3 Omron D6T1A01



Abb. 3.10: Omron D6T1A01

Der Omron D6T1A01 gehört zur Gruppe der D6T Thermalsensoren. Alle D6T Thermalsensoren bestehen aus einer Platine, einer Siliziumlinse, einem Thermopile-Sensor und einem speziellen Adapter. Dabei bündelt die Siliziumlinse die infrarote Strahlung, welche von umliegenden Objekten abgestrahlt wird, auf den unter ihr liegenden Thermopile-Sensor. Dieser wandelt diese infrarote Strahlung dann in elektromotorische Kraft um, aus welcher dann im folgenden die

Temperaturwerte der Objekte berechnet werden. Diese Werte können dann über den Adapter via I2C erhalten werden.

Das I2C-Protokoll ist bei der Gruppe der D6T Thermalsensoren unterschiedlich. Bei dem ausgewählten Typ, dem D6T1A01 beginnt das Protokoll (Abb.3.11 und Abb.3.12) wie gewöhnlich mit dem Start-Bit und der Geräteadresse des Sensors. Anschließend folgt das Write-Bit, welches im Folgenden durch ein ACK des Slaves bestätigt wird. Danach wird das Register angegeben, welches für die spätere Read-Operation ausgelesen werden soll. Beim D6T1A01 ist dieses das 0x4C. Als nächster Teil des Protokolls steht die Repeat Start Condition (Sr) an. Durch diese kann eine Write- und Read-Operation in einem Protokollablauf stattfinden. Der erwähnte Read-Befehl folgt nach der erneuten Geräteadresse mit dem dazugehörigen ACK. Nun stehen die angeforderten Daten in Form von zwei PTAT Byte (Low und High), zwei weitere Byte für das P0 und ein Byte für das PEC zur Verfügung. PTAT steht hierbei für die gemessene Umgebungstemperatur, die als Referenz genommen werden kann und P0 ist beim D6T1A01 das gesamte Field of View (FOV). Hinter jedem dieser Bytes wird ein ACK zur Empfangsbestätigung gesendet. Das Protokoll wird vom bekannten Stopp-Bit (NACK) abgeschlossen. Generell bestehen PTAT und P0 aus zwei Byte. Um aus diesen Bytes einen realen Temperaturwert zu erhalten, müssen diese miteinander verrechnet werden:

$$PTAT = \frac{256 \cdot PTAT(Hi) + PTAT(Lo)}{10} \quad (3.1)$$

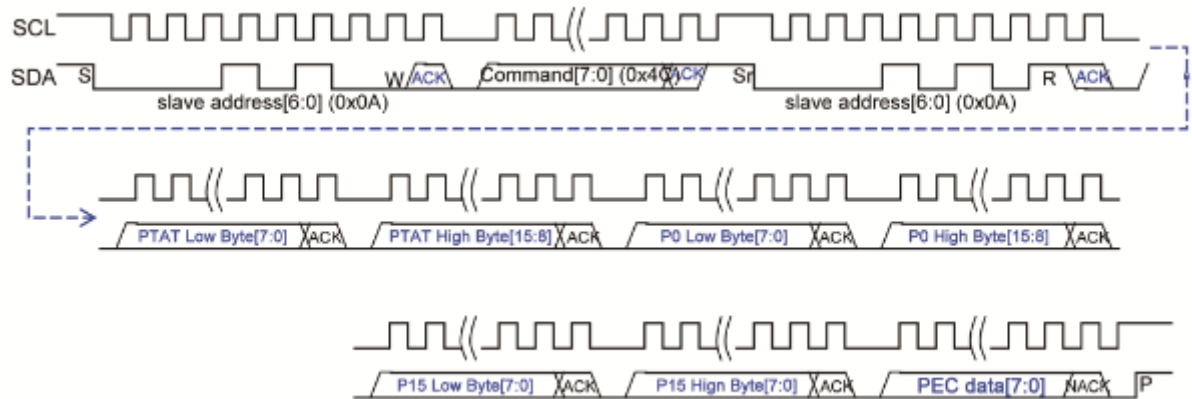
$$T0 = \frac{256 \cdot T0(Hi) + T0(Lo)}{10} \quad (3.2)$$

Das PEC-Byte dient lediglich zur Fehlererkennung.



Abb. 3.11: D6T1A01 Protokoll

Signal Chart



(The D6T-1A-01/-02 models lack P1 through P15)

Fig. 16. Signal Terminal Flow (D6T-1A-01/-02/44L-06)

"S"	: Start Condition
"Sr"	: Repeat Start Condition
"P"	: Stop Condition
"W/R"	: Write (Lo) / Read (Hi)
"ACK"	: Acknowledge reply
"NACK"	: No-acknowledge reply

* Refer to the I2C bus specifications for the definitions of these I2C terms.

Abb. 3.12: D6T1A01 Signalverlauf

[40, 41]

3.4 MQTT

Das MQTT-Modell ist ein Client-Server-Protokoll, bei welchem bidirektional, d.h. vom Client zum Server und vom Server zum Client, kommuniziert werden kann. Das Modell besteht aus zwei verschiedenen Gerätearten, dem Client und dem Broker. Es basiert auf Themen (Topics), welche die Clients abonnieren können (to subscribe). Clients können beide Rollen, die des Empfängers und die des Senders, erfüllen. Eine Art Mittelsmann ist dabei der Broker, welcher Daten zu einem Thema empfängt und diese dann für andere Clients, die dieses Thema abonniert haben, zur Verfügung stellt (Publish). In der Abb. 3.13 ist das erklärte Modell bildlich veranschaulicht:

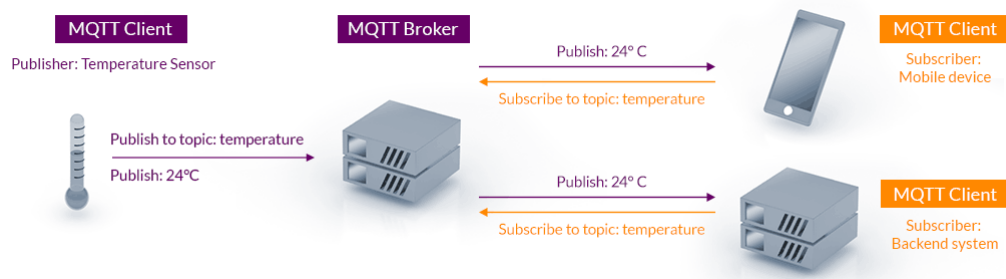


Abb. 3.13: MQTT Modell

Im MQTT System sind 3 Quality of Service (QoS) -Level definiert. Diese unterscheiden sich nach der Zuverlässigkeit über die Ankunft der gesendeten Nachricht. Level 0, at most once, ist nicht sehr zuverlässig, da hier nur die Nachricht versendet wird und nicht geprüft wird, ob diese auch vom Broker empfangen wurde. Bei Level 1, at least once, wird vom Client die Nachricht solange immer wieder verschickt, bis ein ACK vom Broker bei Client wieder ankommt. Hierbei ist zwar gesichert, dass der Datenaustausch funktioniert hat, jedoch wird die Nachricht vielleicht öfter, als dies benötigt ist, verschickt. Bei Level 2, exactly once, wird während der Kommunikation ein Handshake zwischen Client und Server ausgeführt. Hierbei können Client und Server anhand der Handshakeprozedur sicher sein, dass die Nachricht tatsächlich angekommen ist. [42, 43, 44]

3.5 Matter

Matter ist ein Verbindungsstandard, welcher in erster Linie über eine lokale Verbindung im Heimnetzwerk funktioniert. Dabei werden Technologien wie Ethernet, Wi-Fi oder Thread genutzt, wobei die Verbindung mit Thread (3.5.2) noch einmal etwas besonderer ist. Ein großer Vorteil dieses Verbindungsstandards ist nicht nur die Vereinigung vieler verschiedener Standards, wie z.B. Amazon, Google oder anderer SmartHome-Assistenten, sondern die, im selben Zug erreichte, Trennung von der Cloud hin zur **lokalen Kommunikation** ohne das Internet. Dazu wird nur ein matter-kompatibles Gerät benötigt, welches für Apps und SmartHome-Steuerzentralen erreichbar ist. [45] Das grundlegende **Sicherheitsprinzip** bei Matter gleicht dem aus dem Internet. Dieses wird Public Key Infrastructure-Verfahren (PKI) genannt. Anhand von gültigen Zertifikaten, die durch die Hersteller vergeben werden (DAC), werden Verbindungen aufgebaut. Bei diesen ist theoretisch das Aufzeichnen durch Dritte möglich, jedoch können diese das Aufgezeichnete, aufgrund des digitalen Schlüssels, nicht entschlüsseln. Wird ein Gerät in Betrieb genommen sendet dieses automatisch Signale aus, mit welchen es anzeigt, dass es ins Netz-

werk aufgenommen werden möchte. Der Controller startet bei Erhalt des Signals einen PASE-Prozess (Password Authenticated Session Establishment) mit dem neuen Gerät, welches durch einen gerätespezifischen Setup-Code (z.B. einen QR-Code) aufgrund der nötigen Authentifizierung, als sicher gilt. Matter folgt dem Prinzip Security by Design, welches besagt, dass die Kommunikation zwischen Geräten immer, also auch von Anfang an bei der Inbetriebnahme, verschlüsselt sein muss. Nach der Prüfung des DAC erhält das Gerät einen digitalen Ausweis, das OpCert, ein Stammzertifikat (Root Certificat), den digitalen Schlüssel und eine Liste aller Funktionen (ACL). Bei gleichem Stammzertifikat vertrauen sich die Geräte gegenseitig und die ACL gibt dem Gerät an, welche anderen Geräte es steuern oder von welchen es gesteuert werden darf. Des Weiteren gibt es ein digitales Register, das Distributed Compliance Ledger (DCL) (zu deutsch: verteiltes Hauptbuch für Konformität). Es ist ein Netzwerk aus unabhängigen Servern, die entweder unternehmensintern oder durch das CSA (Connectivity Standard Alliance) betrieben werden. Ein wichtiger Punkt hierbei ist die Dezentralisierung, die vor Angriffen auf das Netzwerk schützt und die Kommunikation untereinander, die über ein kryptografisch gesichertes Protokoll abläuft. In dem Register werden Daten, wie z.B. Informationen zum Anbieter, Liste der Stammzertifikate, IDs, Informationen zur aktuellen Softwareversion, etc., gespeichert. Um ein Update für ein Produkt bereitzustellen oder sogar ein neues Gerät auf den Markt zu bringen, gibt es spezielle Prüfinstitute. Bei erfolgreichem Zertifizierungsprozess durch das Institut, wird die CSA benachrichtigt, die das Produkt oder das Update dann als zertifiziert eintragen kann. Anhand von diesem Status können die verschiedenen Matter-Ökosystem (z.B. Android, Apple, etc.) dann bedenkenlos das Gerät zu ihrem System hinzufügen. [46] Ein weiterer Vorteil von Matter nennt sich **Multi Admin**. Multi Admin bedeutet, dass ein Gerät in verschiedenen Matter Fabrics (nach Standard: mindestens 5 gleichzeitig möglich) installiert werden kann. Das heißt z.B., dass nicht jede Person in einem Haushalt das selbe Ökosystem benutzen muss. Ein Licht oder eine Steckdose kann somit über mehrere verschiedene Systeme gesteuert werden. [47] Das, im Kontext von Multi Admin erwähnte, **Matter Fabric** ist ein aufgespanntes Mesh-Netz von Matter Nodes. Eine oder auch mehrere (bei Unterfunktionen eines Gerätes) Nodes sind physikalische Geräte, die das Mesh-Netz bilden. Durch Erhalt eines eindeutigen Sicherheitszertifikates, das nur für dieses eine Fabric gilt, wird eine Node in dieses aufgenommen. Dieses Zertifikat erhält das Node durch den Commissioner. Ein **Matter Commissioner** richtet das neue Node ein bzw. gibt ihr Informationen mit und weist sie ins neue Fabric ein. Der Commissioner ist oft mit dem Matter Controller in einem Gerät kombiniert. Ein **Matter Controller** ist für die Steuerung installierter Geräte zuständig. Ein Controller kann vieles sein, z.B. eine App,

ein Smart-TV oder auch spezielle SmartHome-Zentralen. Dabei muss es nicht nur einen Controller geben, sondern man kann von mehreren Geräten aus in einem Matter-Fabric die Nodes ansteuern bzw. kontrollieren. [48, 49]

3.5.1 Weitere Begriffe

Das Konzept des **Matter Binding** ist ein großer Schritt zur Automatisierung im Smart Home Bereich, speziell in der Kategorie der Funkkommunikation. Hierbei gehen Geräte wie z.B. ein Temperatursensor und eine Heizung eine Verbindung ein, durch welche die Heizung die Werte des Sensors direkt durch eine Art Abonnementmodell (siehe MQTT) erhält. Anhand von zuvor definierten Richtwerten zur Temperatur in einem Raum, könnte sich die Heizung somit selbst regeln. Leider haben die meisten Ökosysteme bisher noch keine Bindings in ihren Systemen vorgesehen oder es fehlt an Bedienelementen mit geeigneten Einstellungen für die entsprechenden Bindings. [50]

Der Begriff **Matter Casting** stammt von Amazon. Amazon benutzt ihn als Synonym für Medien-Streaming mit Prime Video. Man kann das Smartphone durch Drücken des Casting Symbols einfach mit dem Endgerät verbinden. Dabei kann man sich Infos zur aktuellen Serie / Film über das Smartphone anzeigen lassen oder dies einfach als eine Art von Fernbedienung benutzen. Man kann sogar die App auf dem Smartphone nach Starten der Wiedergabe schließen, da das Endgerät mit dem Internet verbunden ist. Beim Thema Matter Casting wird außerdem sehr zukunftsweisend gedacht. Es soll möglich sein nicht nur Videos, sondern auch noch Benachrichtigungen, wie z.B dass die Waschmaschine fertig ist oder dass der Feuermelder ausgelöst hat, anzeigen zu lassen. [51]

Ein weiteres, bisher noch nicht erwähntes Gerät, ist die **Matter Bridge**. Sie hat die Aufgabe Geräte, die nicht dem Matter Standard entsprechen, in das Matter-Netzwerk einzubinden. Die Matter Bridge ist also eine Art Übersetzer, der die fremden Systeme matterfähig macht. [52]

3.5.2 Thread-Protokoll

Thread ist ein Funkprotokoll, welches im Matter-Standard als Alternative zum WLAN vorgesehen ist. Dabei bietet es zwei Vorteile: Das Protokoll benötigt wenig Energie zum Empfangen und Versenden von Daten und ist somit sehr stromsparend. Des Weiteren ist Thread Mesh-fähig. Das bedeutet, dass die mit der Stromleitung verbundenen Geräte als Repeater fungieren und sie somit das Netzwerk flächenmäßig vergrößern, wodurch auch weiter entfernte Geräte eingebunden werden können. Die mit der Stromleitung verbundenen Geräte werden im

Thread-Netzwerk als **Router** bezeichnet. Sie machen das Mesh-Netzwerk robuster und stabiler. Beispiele für Router sind z.B. Lampen oder Zwischenstecker an Steckdosen. Neben den **Endgeräten** gibt es noch als 3. Gerätekategorie den **Border-Router**. Von diesem Gerätetyp wird in jedem Thread-Netzwerk immer mindestens ein Exemplar benötigt. Sie sind die Verbindung zum IP-Netzwerk, wodurch das Thread-Netzwerk dann via Internet oder über Steuerung per Smartphone zugänglich wird. Die Router und Endgeräte erhalten eine IP-Adresse (IPv6), was sie leichter adressierbar macht. [53, 54]

4 Durchführung

4.1 Aufbau auf dem Breadboard

Bevor man mit dem eigentlichen Programmcode beginnen kann, muss man zuerst die Hardware aufbauen und korrekt anschließen. Hierfür standen zusätzlich, neben den Sensoren und dem ESP32-C6 Board, ein Breadboard, Jumper-Wires und verschiedene Widerstände bereit. Zuerst wurde der ESP Controller auf das Board gesteckt, wobei zu beachten war, dass jeweils auf beiden Seiten des Boards, an den Anschlusspins eine Reihe Platz gelassen werden musste, um mögliche Anschlüsse zu legen.

Anhand des Layouts [55] kann man den Grundaufbau und die speziellen Funktionen der einzelnen Pins erkennen. Da es sich bei jedem der 3 verwendeten Sensoren um I2C-fähige Sensoren handelt, liegt es nahe eine 'SDA-Kette' und eine 'SCL-Kette' mit den Sensoren zu bilden.

Als SDA-Pin wurde der GPIO-Pin 18 und als SCL-Pin wurde der GPIO-Pin 19 gewählt. Von diesen Pins liegt jeweils eine Leitung zu einer neuen Breadboard-Reihe, in welcher die jeweiligen Sensoren der entsprechenden Leitungsart angeschlossen sind (I2C-Bussystem). Dabei ist zu beachten, dass noch 2,2 kOhm Widerstände zwischen den Reihen und der VCC-Verbindung des HYT 271 und des Joy-It CCS811 liegen, welche als Pull-up Widerstände fungieren. Zudem liegen von dem Ground-, dem 3V- und dem 5V-Pin diverse Leitungen zur Spannungsversorgung. Die 3V-Spannungsversorgung wird für den HYT 271 und den Joy-It CCS811 benötigt, während die 5V-Spannungsversorgung für den Omron D6T1A benötigt wird. In der Abb. 4.1 ist der komplette Aufbau auf dem Breadboard dargestellt:

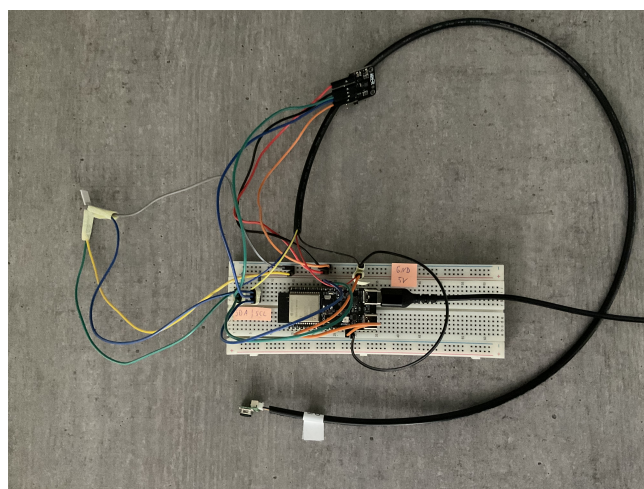


Abb. 4.1: Hardwareaufbau

4.2 Einrichtung VS Code mit der ESP-IDF

Für die grundlegende Umgebung zum Schreiben des Programms wurde VS Code und die zum ESP32C6 benötigte Software ESP-IDF genutzt [56]. Zuerst wurde das WSL-System (Windows Subsystem for Linux) aufgesetzt. Hierzu wurde die offizielle Seite von Espressif genutzt. Dabei ist zu erwähnen, dass die Version Ubuntu-22.04 benutzt wurde [57]. Ebenso wurden die grundlegenden Schritte des Standard Toolchain Setups für diese WSL durchgeführt [58]. Um nun das erforderliche Setup zu komplettieren, wurde ESP-Matter konfiguriert [59]. Zum Testen des Programms muss der ESP32C6 auch noch an die WSL angeschlossen werden. Hierzu wurde USBIPD genutzt [60].

Außerdem wurde die Flashgröße von 2MB auf die möglichen 8MB beim ESP32C6 angepasst.

4.3 Matter Sensors

Als Grundgerüst des Projekts wurde die Vorlage aus dem ESP-Matter Ordner 'esp-matter/examples/sensors' [61] benutzt. Das Sensor-Projekt zielte darauf ab zwei Sensoren, einen SHTC3- und einen PIR-Sensor, einzubinden. Im Folgenden werden die Änderungen am Projekt dargestellt und die wichtigsten Bausteine erläutert.

4.3.1 Aufbau

Der anfängliche Aufbau des Projekts wird in der folgenden Abbildung kurz dargestellt:

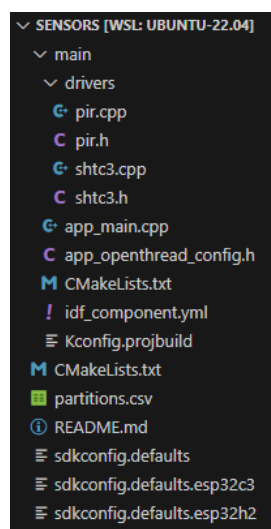


Abb. 4.2: Sensors-Struktur

In den grundlegenden Dateien, wie z.B. CMakeLists, der idf_component oder der app_openthread_config wurden keine Änderungen vorgenommen.

4.3.2 Drivers

Da der in der Vorlage verwendete Sensor, der SHTC3, über ein I2C-Protokoll kommuniziert, wurde sich im Folgenden, bei dem Erstellen der neuen Dateien, an den beiden bestehenden Dateien shtc3.cpp und shct3.h orientiert. In der neuen .h-Datei werden dann zuerst drei 'Callbacks' für die drei Sensoren erstellt. Im Anschluss werden die Strukturen der Sensoren mit ihren jeweiligen Parametern, wie z.B. beim HYT271 Luftfeuchtigkeit und Temperatur, definiert. Abschließend werden die drei 'init' Funktionen, die in der .cpp-Datei definiert sind, und die für den CCS811 benötigte write-Funktion noch deklariert.

Als Nächstes wird auf die Änderungen in der .cpp-Datei eingegangen. Zu Beginn werden die Strukturen aus der .h-Datei um einen Timer erweitert. Des Weiteren wird der I2C-Master mit der 'i2c_config_t' konfiguriert. Im Anschluss sind zwei Funktionen, write und read, definiert, mit welchen auf die Sensoren über I2C zugegriffen werden kann. Der Aufbau der beiden Funktionen ist, vor allem beim write, auf die Sensoren angepasst. Die weitere Struktur der Datei beinhaltet Funktionen für die Sensoren, die verschachtelt aufgerufen werden. In ihnen kommen unter Anderem auch die write- und read-Funktion zum Einsatz. Diese Funktionen sind wie folgt aufgebaut:

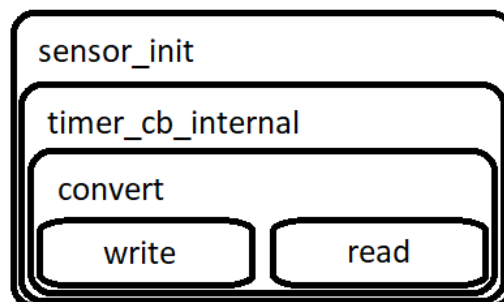


Abb. 4.3: I2C Struktur der Funktionen

In der convert-Funktion werden die write- und read-Funktion mit den spezifischen Adressen und Registern der Sensoren ausgeführt und die ausgelesenen Daten in einem array gespeichert. Diese Daten müssen dann anhand einer sensorspezifischen Umrechnung, welche aus den Datenblättern (Kapitel 3.3) hervorgeht, in das entsprechende Format der gemessenen Variable konvertiert werden. Im timer_cb_internal werden zum einen ein Zeiger (arg) erstellt, der in einen

`_cxt_t *`-Typ umgewandelt wird und somit auf die Konfiguration zugreifen kann. Des Weiteren wird ein Callback definiert, welchem die drei Argumente, `endpoint_id`, der jeweilige errechnete Wert und die `user.data`, zugewiesen werden. In der letzten Funktion, der `init`, wird der Sensor initialisiert und die periodische Abfrage gestartet. Hierzu muss zuerst die I2C-Schnittstelle initialisiert werden. Das Argument der Funktion wird unter der `.config` des Sensors abgespeichert. Im Anschluss wird der Timer erstellt. Dabei wird die `timer_cb_internal`-Funktion dem `.callback` zugewiesen, also immer dann ausgeführt wenn der Timer ausläuft. Zudem wird ein Zeiger auf den Sensor als Argument übergeben, sodass die Callback-Funktion auf die Konfiguration des Sensors zugreifen kann. Des Weiteren gibt es noch, spezifisch für den CCS811, eine `write`-Funktion, mit der im entsprechenden `sensor_init` der Modus gesetzt und der `APP_START` ausgeführt wird.

4.3.3 `app_main`

Die `app_main` besitzt fünf `sensor_notification`-Funktionen, Temperatur, Luftfeuchtigkeit, `eCO2`, `TVOC` und Anwesenheit. Bei diesen werden jeweils die drei Argumente aus der `timer_cb_internal`-Funktion benötigt.

Das Attribut 'MeasuredValue' des jeweiligen Clusters wird im entsprechenden Endpunkt gespeichert. Nur bei der Anwesenheitsbenachrichtigung ist der 'MeasuredValue' durch 'Occupancy' ersetzt, da die Anwesenheit in einem bool mit wahr oder falsch gespeichert wird. Mit dem `factory_reset_button_register` wird, wie der Name schon sagt, ein Reset-Taster, der alles auf Werkseinstellungen zurücksetzt, registriert. Mit der nächsten Funktion, der `open_commissioning_window_if_necessary`, wird ein Commissioning Fenster geöffnet, falls noch keine Verbindung zu einem Fabric besteht und somit ein Controller das Gerät neu ins Netz einbinden kann. Der `app_event_cb` ist die zentrale Callback-Funktion von Matter. Mit den Events `CommissioningComplete`, `FailSafeTimerExpired`, `FabricRemoved` und `BLEDeinitialized` kann der Nutzer den aktuellen Status seines Gerätes einsehen. Die Funktion `app_identification_cb` identifiziert den Callback anhand der gegebenen Parameter wie z.B. `type` oder `endpoint_id`. Als Letztes gibt es noch die Funktion `app_attribute_update_cb`, welche alle Attribute anhand der gegebenen Parameter updatet.

In der 'extern 'C' void `app_main`' wird zuerst das NVS (Non-Volatile Storage) initialisiert. Der oben genannte (Hardware-)Reset-Button wird ebenfalls registriert. Folgend wird eine zentrale Node und fünf weitere Nodes für die fünf, oben schon genannten, Variablen erstellt. Bei diesen ist der Aufbau im Grundlegenden der Gleiche: eine entsprechende config (`esp_endpoint`-Dateien) wird verwendet um danach einen Endpunkt zu erstellen. Im Weiteren werden die drei

Sensortreiber mit den eben angesprochenen Endpunkten und der oben genannten Benachrichtigung initialisiert. Nur bei dem Präsenzsensord ist noch der Sensortyp und dessen bitmap spezifiziert. Im Folgenden werden bei den drei Sensoren die fünf Callback-Funktionen mit den Endpunkten konfiguriert. Abschließend wird Matter gestartet.

Nach erfolgreichem Setup (4.2) und 'build' ist in der WSL für die Befehle 'flash' und 'monitor' folgende Ausgabe zu sehen: A.1.

4.4 PCB

Aus dem Breadboardaufbau (3.1), welcher vor allem für Testzwecke sehr gut geeignet ist, soll eine PCB entstehen. Diese ist dann im Vergleich zum Testaufbau strukturierter und geeigneter für einen realitätsnahen Einsatz. Ebenso kann aufgrund kürzerer Leiterbahnen im Vergleich zu den Jumper-Wires der Energieverbrauch, wenn auch nur leicht, optimiert werden. Zudem sind die Leitungsverbindungen aufgrund des erstellten Plans und der Label gut nachvollziehbar.

Das verwendete Programm zur Erstellung der PCB ist EasyEDA.

Zur Erstellung einer PCB gehören mehrere Schritte. Im 1. Schritt wird der grundlegende Plan mit den Komponenten erstellt, auf dem diese durch Anschlusslabel oder auch direkt miteinander verbunden werden. Dabei ist darauf zu achten, dass im Sinne der zukünftigen Erweiterbarkeit der Platine wenige Anschlüsse unbelegt bleiben. Aus diesem Grund werden sogenannte Pin Header verwendet, mit welchem freie Anschlüsse verbunden werden können. Folgend ist der Anschlussplan des Pin Headers zu sehen:

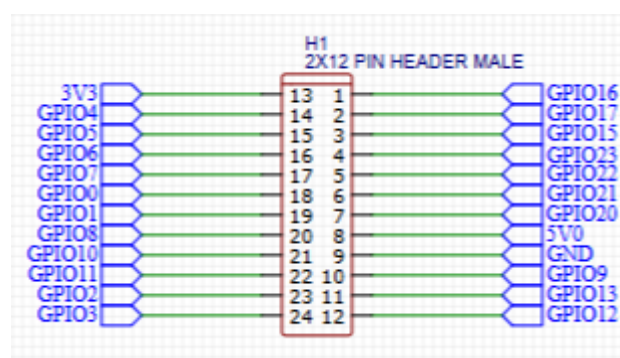


Abb. 4.4: Pin Header

Zu jeder Komponente muss außerdem ein Footprint, welcher die genauen Abmaße des Bauteils, vor allem die Abstände und Dimensionen zwischen den einzelnen Pads oder Durchgangslöchern, beschreibt, erstellt werden. Anhand dieser Footprints und der angegebenen Verbindungen im zuvor erstellten Schaltplan kann dann ein Entwurf der Leiterplatte erstellt werden.

Wichtig zu beachten ist hierbei die Anzahl an verfügbaren Layern. In diesem Fall wurde eine 2-Layer PCB entworfen.

Um die Bauteile auch korrekt mit den Leiterbahnen zu verbinden, wenn es keine Möglichkeit zur Durchkontaktierung gibt, ist es wichtig, dass alle Verbindungen mit den Pins der Bauteile auf dem unteren Layer der Platine erfolgen. Dort können dann die Lötunkte gesetzt werden. Aufgrund der hohen Dichte der Leiterbahnen auf der Platte, muss bei mehreren Verbindungen immer wieder das Layer mithilfe von Vias gewechselt werden. In Abb. 4.5 ist das Layout der fertigen Platine zu sehen:

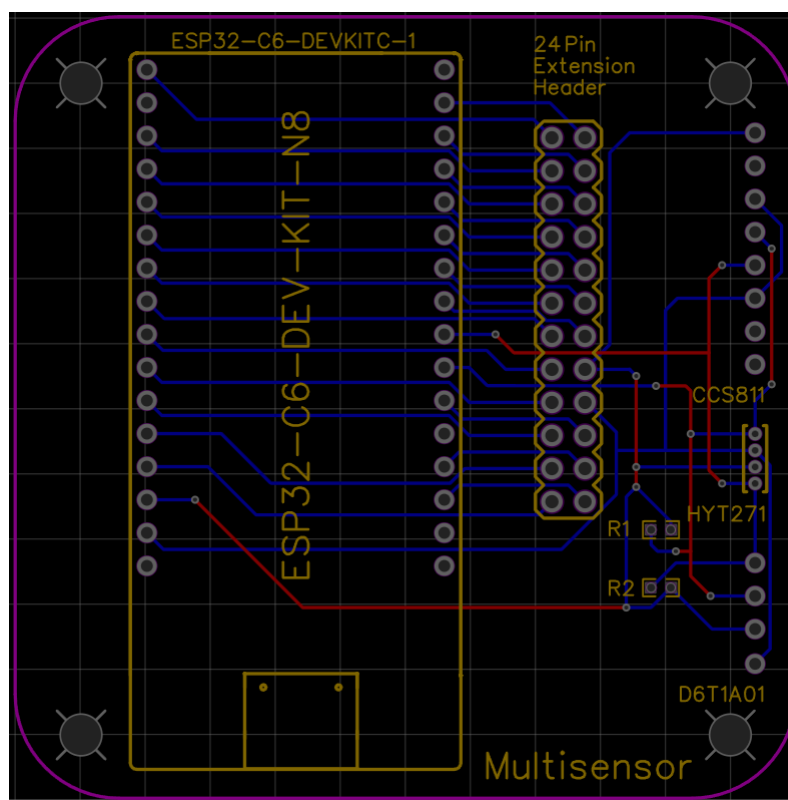


Abb. 4.5: PCB Layout

Für die Fertigung in der Elektrowerkstatt wird eine .board Datei benötigt. Um diese von EasyEDA umzuwandeln, müssen einige Schritte durchgeführt werden. Zuerst muss diese über die Option Export zu einer Altium Datei konvertiert werden. Anschließend wird diese dann im Altium Designer zu einer P-Cad ASCII Datei umgewandelt, welche dann wiederum final von EAGLE in eine .board Datei importiert werden kann.

Mithilfe einer eigenen Lötstation wurden die Hardware-Komponenten mit der angefertigten Platine verlötet.

4.5 SmartHome Server

Als SmartHome-Server wurde HomeAssistant ausgewählt. Dabei wird der Server via VirtualBox gehostet. VirtualBox und das dazugehörige Extension Pack (jeweils Version 7.1.8) [62] wurden zusammen mit der zugehörigen Imagedatei heruntergeladen, installiert und aufgesetzt [63].

Nachdem das Image via VirtualBox gestartet wird, kann der Server im Browser unter `http://homeassistant.local:8123/` im Heimnetzwerk erreicht werden. Hier hat man bei der Option 'Gerät hinzufügen' die Möglichkeit als Anbieter Matter zu wählen. Dafür ist ein QR-Code bzw. eine Passphrase erforderlich.

Ein solcher QR-Code kann unter der Dokumentation der SDK für ESP mit Matter unter Commissioning gefunden werden [59]. Mit diesem konnte erfolgreich eine Verbindung über Wi-Fi hergestellt werden, wodurch anschließend der Multisensor unter dem Reiter 'Geräte' angezeigt wurde. Hier sind die wichtigsten Daten des Sensors und Möglichkeiten zur Konfiguration zu finden:

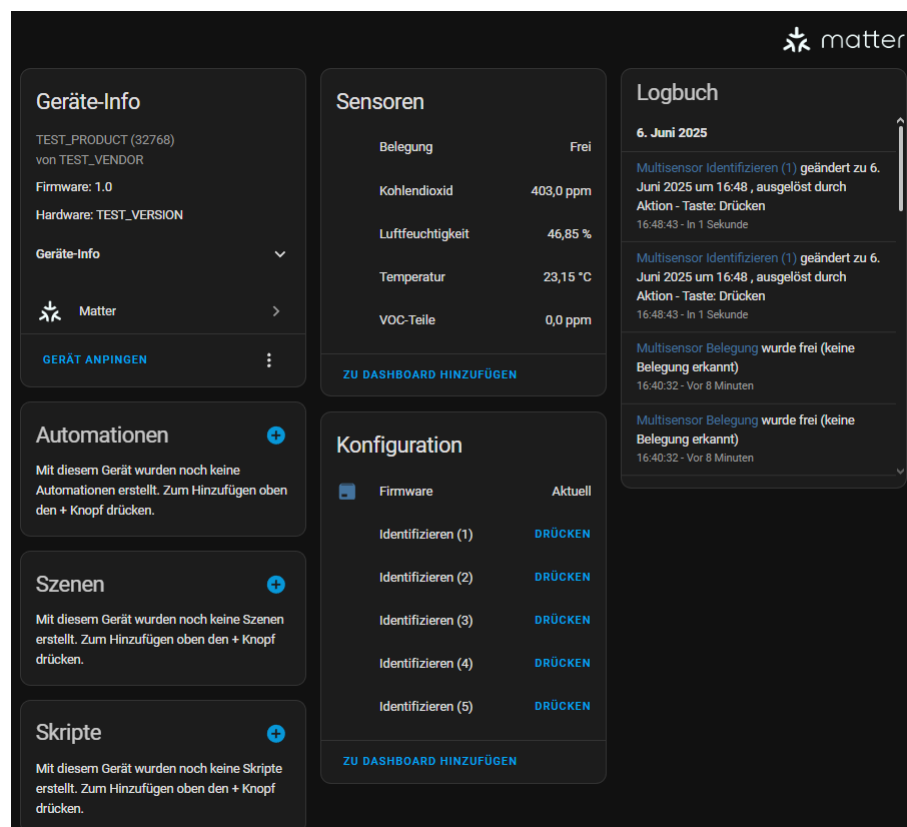


Abb. 4.6: Multisensorgerät in HomeAssistant

Durch die Einbindung des Servers in das Projekt liegt nun folgende Kommunikationsstruktur vor:

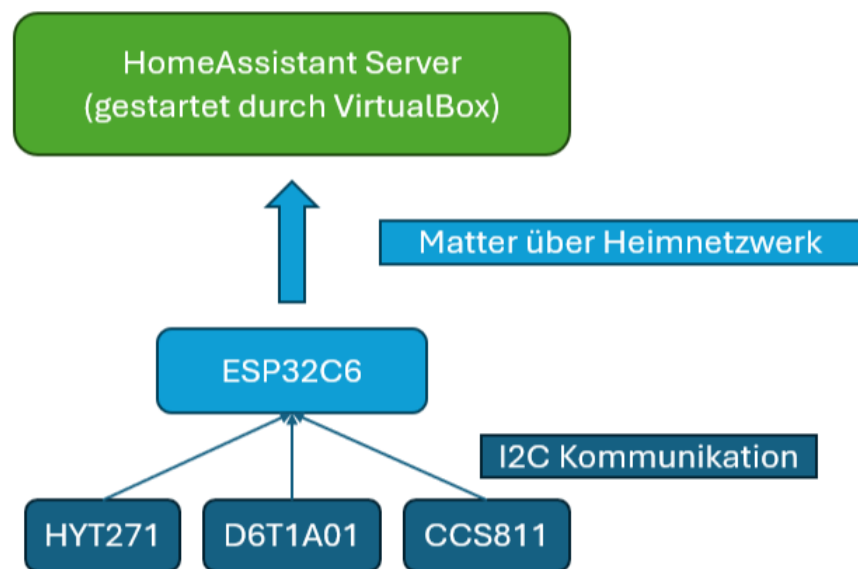


Abb. 4.7: Projektstruktur Kommunikation

4.6 Anwendung

Der Multisensor besitzt die Möglichkeit wichtige Luftdaten, wie die Temperatur, die Luftfeuchtigkeit und die Luftqualität, und eine Präsenzerkennung zu messen. Im Folgenden werden die Einsatzgebiete der Sensoren und die möglichen Automatisierungen mit anderen SmartHome Geräten dargestellt: Durch die Temperaturmessung kann die Abweichung von der gewünschten Soll-Temperatur eines Raumes erkannt werden. Diese kann dann durch manuelles oder automatisiertes Einschalten von einer Heizung zum Erhöhen oder einer Klimaanlage zum Erhöhen oder auch zum Senken der Temperatur eingestellt werden. Mit der Luftfeuchtigkeitsmessung kann ebenfalls ein Vergleich zu einem Soll-Wert gezogen werden. Mit entsprechenden Raumbe- oder Raumentfeuchtern kann der Wert geregelt werden. Die Luftqualität zeigt die Luftverschmutzung bzw. die Reinheit der Luft in einer Umgebung an. Innerhalb eines Raumes können dann Luftfilter zur automatisierten Reinigung geschaltet oder auch Fenster manuell geöffnet werden. Die Präsenzmessung kann ein Objekt, aufgrund eines Vergleich der Temperatur des Objektes mit der Umgebungstemperatur, auf eine kurze Distanz (max. 50cm) erkennen. Diese Eigenschaft kann in vielen Gegebenheiten verwendet werden, aber vor allem wird sie oft mit Ein- und Ausschalten von Lichtquellen in Verbindung gebracht.

Ein spezieller Einsatz des Multisensors kann z.B. in einer Büroumgebung an einem Schreibtisch gefunden werden. Dabei können die Luftdaten im Büro gelesen werden und je nach fest-

gelegten Werten kann automatisiert oder auch manuell gesteuert werden. Die Präsenzmessung kann in Kombination mit der Uhrzeit dazu genutzt werden um Licht am Platz einzuschalten, wenn eine Präsenz erkannt wird. Des Weiteren kann diese auch dazu benutzt werden den PC bei längerer Pause, also ein nicht Vorhandensein einer Präsenz, in einen Energiesparmodus zu schalten oder gar nach Arbeitsende bei Vergessen komplett auszuschalten.

Hierzu müsste die Platzierung des Sensors zentral auf dem Schreibtisch z.B. hinter der Tastatur und unter den Bildschirmen erfolgen. Dabei ist der Präsenzsensor noch einmal aufgrund der angeschlossenen Leitung variabel einstellbar.



Abb. 4.8: Einsatzbeispiel

In Abb 4.8 wurde der Präsenzsensor behelfsmäßig an einem Dreibeinstativ einer Kamera befestigt.

5 Ergebnisse und Auswertung

Die grundlegende Kommunikation, die Auslesung und Verarbeitung der Sensorwerte, mit anschließender Weiterleitung des ESPs an den SmartHome Server via Matter, funktioniert. Ebenfalls wurde das esp_matter Matter Repository um weitere Sensortypen für die Messung der Luftqualität erweitert.

Alle geplanten Verbindungen auf der PCB konnten erfolgreich mit einem Multimeter auf Durchgang geprüft werden. Die Funktion der PCB kann anhand der passenden Messwerte festgestellt werden. Diese werden nach Auslesen der Sensoren und Verarbeiten durch die Software in erwarteter Form auf dem selbst gehosteten Server angezeigt.

Die Reichweite bzw. auch die damit zusammenhängende Genauigkeit der Messung des D6T1A01 ist kürzer bzw. schlechter als im Datenblatt angegeben, was die Möglichkeiten des Gebrauchs einschränkt. Diese ist bei nur knapp 30 cm einzuordnen. Wenn das zu messende Objekt nicht komplett den Field of View (FOV) bedeckt spielt beim Omron D6T1A01 die restliche Temperatur des Hintergrunds im FOV auch eine Rolle. Zur Errechnung des aufgenommenen Wertes wird dann der Durchschnitt des Sichtfeldes genommen. Dadurch wird das Objekt nicht immer erkannt.

6 Schlussfolgerung und Ausblick

Die grundlegende Funktion des Multisensors wurde erfolgreich geprüft. Das Kommunikationsmodell zwischen den Sensoren, dem ESP und dem Server funktioniert.

Bei der PCB und der Änderung des esp_matter Repositorys besteht noch Optimierungspotential. Dabei könnten bei der PCB die Lochabstände der Widerstände und die Lochgröße für den ESP optimiert werden. Um das Problem mit der Lochgröße des ESPs zu lösen mussten Buchsenleisten verwendet werden. Hier ist eine potentielle Verbesserung des Entwurfes zu sehen:

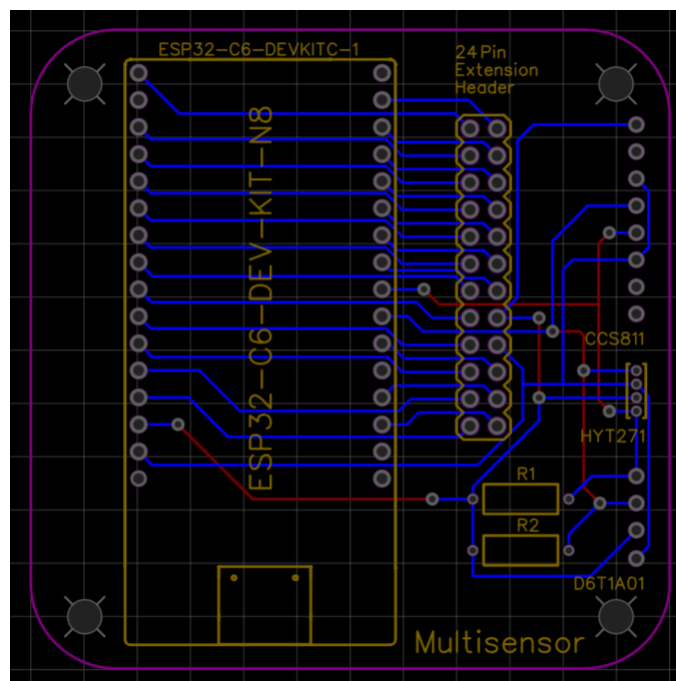


Abb. 6.1: PCB Verbesserung (EasyEDA)

Die Änderung des Repositorys beinhaltet eine noch nicht eingefügte Funktion, die im Verlauf der Arbeit durch ein Update hinzugefügt wurde.

Da der D6T1A01 Präsenzsensoren nur eine kurze Reichweite aufweist, kann dieser für weniger Anwendungen verwendet werden. Eine höhere Reichweite kann durch einen anderen Sensor mit nicht nur einem Feld (1x1), sondern mit mehreren Feldern, also einer Matrix, erzielt werden. Hier könnte dann z.B. der D6T44L06 verwendet werden, der eine Matrix von 4x4 Feldern vorweisen kann. So könnte die Temperatur und Position des Objektes / der Person genauer erkannt werden. Dadurch wäre der Multisensor auch für weitere Anwendungen geeignet: z.B. die Erkennung einer Person in einem Raum, bei Montage des Sensors an der Decke, zum

Einschalten eines Lichtes. Falls dies gewünscht ist, ist es durch die kabelgebundene Verbindung, durch welche auch die genaue Einsatzposition variiert und individuell angepasst werden kann, auch möglich den Sensor auszutauschen. Dazu müssten nur kleine Änderungen am Code, an der Datei `ccs811_hyt271_d6t1a01_i2c.cpp`, genauer an den Funktionen `d6t1a01_convert` und `d6t1a01_timer_cb_internal`, vorgenommen werden. Diese resultieren aus der Anzahl an benutzten Felder die von einem Feld auf fünfzehn Felder ansteigt. Außerdem ist zur D6T Reihe zu sagen, dass diese nur einwandfrei funktionieren, wenn das zu erkennende Objekt eine Differenz zur gemessenen Referenztemperatur aufweist. Vor allem an heißeren Tagen führt dies zu Problemen.

Des Weiteren wäre es sinnvoll ein Gehäuse für den Multisensor zu errichten. Dies kann durch einen 3D-Drucker mit einem dazugehörigem Programm gedruckt werden. Dabei gilt es auf verschiedenste Punkte zu achten:

Die zwei Sensoren, die direkt mit der Platine verlötet sind, der CCS811 und der HYT271, sollten zwar vom Gehäuse geschützt werden, jedoch auch noch in der Lage sein, korrekte Messwerte zu liefern. Daher müssen an den Messstellen der Sensoren Aussparungen vorgesehen werden. Der D6T1A01 ist nicht direkt mit der Platine verlötet, aufgrund der Möglichkeit, dass der Sensor bestenfalls flexibel auf den Arbeitsplatz einstellbar sein soll. Hier wäre die Überlegung eine Säule zu errichten, auf welcher der Sensor in einer Art drehbaren Kugel, mit Aussparung für den Sensor, montiert ist. Um die Leitung auf die entsprechende Länge anpassen zu können, müsste diese noch einmal von der Platine gelöst werden. Des Weiteren darf eine Öffnung für den Anschluss zur Versorgung des ESPs nicht vergessen werden. Natürlich wäre es auch wichtig die Anschlüsse des Pin Headers frei zu lassen, um weitere Sensoren zum Projekt hinzufügen zu können.

Hierzu werden im Folgenden noch einige Beispiele aufgelistet: Zur Erweiterung der Messung der Luftqualität könnte ein (Fein-)Staubsensor eingesetzt werden. Dieser kann, anders als der CCS811, welcher gasförmige Substanzen misst, feste oder auch flüssige Teilchen in der Luft messen. Bei Überschreitung eines Grenzwertes könnte, ebenso wie beim CCS811, eine Meldung zum Durchlüften des Raumes oder eine Automatisierung mit einem Luftfilter geschaltet werden. Als weitere Möglichkeit könnte ein Lichtsensor angeschlossen werden. Mit diesem könnte die Helligkeit am Arbeitsplatz kontrolliert werden. Ggf. könnten dann Rollläden automatisch / manuell geöffnet oder auch geschlossen bzw. ein Licht ein- oder ausgeschaltet werden. Bei einem Büro, welches von mehreren Personen benutzt wird, z.B. bei der Schichtarbeit, könnte auch der Einsatz eines RFID-Moduls Vorteile mit sich bringen. Somit kann nicht nur überwacht

werden, welche Person zu welchem Zeitpunkt anwesend war, sondern auch das Büro immer wieder ganz einfach personalisiert werden. Als Beispiele wären hier die Höhe des Schreibtischs, die Raumtemperatur oder auch die Helligkeit im Raum zu nennen.

Alle wichtigen Dateien des Projekts sind unter folgendem GitHub Repository abgelegt: [64]

Literaturverzeichnis

- [1] "Esp32-c6 series datasheet version 1.3." [Online]. Verfügbar: https://www.espressif.com/sites/default/files/documentation/esp32-c6_datasheet_en.pdf [Zugriff am: 2025-05-30]
- [2] "Hardware | arduino documentation." [Online]. Verfügbar: <https://docs.arduino.cc/hardware/> [Zugriff am: 2025-05-30]
- [3] "What open source means & why it's important." [Online]. Verfügbar: <https://www.arduino.cc/education/what-open-source-means-why-it-is-important> [Zugriff am: 2025-05-30]
- [4] "Arduino » alles wissenswerte über arduino." [Online]. Verfügbar: <https://www.conrad.de/de/ratgeber/schule-unterricht/arduino.html> [Zugriff am: 2025-05-30]
- [5] "Raspberry pi hardware - raspberry pi documentation." [Online]. Verfügbar: <https://www.raspberrypi.com/documentation/computers/raspberry-pi.html> [Zugriff am: 2025-05-30]
- [6] S. Safaric und K. Malaric, "Zigbee wireless standard," in *Proceedings ELMAR 2006*. IEEE, 2006, Seite 259-262.
- [7] P. Dhillon und H. Sadawarti, "A review paper on zigbee (ieee 802.15. 4) standard," *International journal of engineering research and technology*, Band 3, 2014.
- [8] admin, "Z-wave | z-wave europe - the leading european distributor for smart home products." [Online]. Verfügbar: <https://zwave.eu/de/zwave/> [Zugriff am: 2025-04-13]
- [9] M. R. Ghorji, T.-C. Wan, und G. C. Sodhy, "Bluetooth low energy mesh networks: Survey of communication and security protocols," *Sensors*, Band 20, Nr. 12, Seite 3590, 2020.
- [10] "Bluetooth technologie Übersicht | bluetooth® technologie website." [Online]. Verfügbar: <https://www.bluetooth.com/de/learn-about-bluetooth/tech-overview/> [Zugriff am: 2025-04-10]
- [11] "Internet of things (IoT) | wi-fi alliance." [Online]. Verfügbar: <https://www.wi-fi.org/internet-things-iot> [Zugriff am: 2025-05-30]

- [12] Y. Guo, S. Zhang, und D. Xiao, "Overview of wi-fi technology," in *2012 International Conference on Computer Application and System Modeling*. Atlantis Press, 2012, Seite 1293–1296.
- [13] B. van Venrooy, "Sicherheit in der heimautomatisierung," Ph.D. dissertation, BSc thesis, Hochschule Bonn-Rhein-Sieg, 2016, homematic-Kapitel (Seiten 43–51).
- [14] Homematic IP: Per funk oder kabel zum smart home. [Online]. Verfügbar: <https://homematic-ip.com/de/das-ist-homematic-ip> [Zugriff am: 2025-07-06]
- [15] Shelly europe. [Online]. Verfügbar: <https://www.shelly.com/de> [Zugriff am: 2025-07-28]
- [16] Shelly compatibility. [Online]. Verfügbar: <https://www.shelly.com/de/pages/compatibility> [Zugriff am: 2025-07-28]
- [17] N. Gleitsmann. Der neue shelly-chipsatz und seine vorteile für ihr smart home. [Online]. Verfügbar: <https://www.reichelt.de/magazin/news/neuer-shelly-chipsatz-fuer-smarte-geraete/> [Zugriff am: 2025-07-28]
- [18] S. E. LTD, "Sicherheitspapier über shelly geräte und ihre infrastruktur." [Online]. Verfügbar: https://cdn.shopify.com/s/files/1/0871/9967/8813/files/SecurityPaper_B2B_EN_Final_1.pdf [Zugriff am: 2025-07-28]
- [19] T. Kröber, "Forensische analyse von vernetzten bewegungssensoren: Untersuchung und dokumentation übertragener daten," Ph.D. dissertation, Hochschule Mittweida, 2024.
- [20] Bosch smart home: Ihr smartes zuhause. [Online]. Verfügbar: <https://www.bosch-smarthome.com/de/de/> [Zugriff am: 2025-07-28]
- [21] Hilfe zu kompatiblen produkten/partnern. [Online]. Verfügbar: <https://www.bosch-smarthome.com/de/de/support/hilfe/hilfe-zur-kompatibilitaet/> [Zugriff am: 2025-07-28]
- [22] Eve | evehome.com. [Online]. Verfügbar: <https://www.evehome.com/> [Zugriff am: 2025-07-28]
- [23] Privatsphäre | evehome.com. [Online]. Verfügbar: <https://www.evehome.com/de/privatsphaere> [Zugriff am: 2025-07-28]
- [24] Produktsicherheit | evehome.com. [Online]. Verfügbar: <https://www.evehome.com/de/security> [Zugriff am: 2025-07-28]

- [25] Über uns | evehome.com. [Online]. Verfügbar: <https://www.evehome.com/de/ueber> [Zugriff am: 2025-07-28]
- [26] Eve – das ist smart home mit 100% privatsphäre | evehome.com. [Online]. Verfügbar: <https://www.evehome.com/de/blog/eve-das-ist-smart-home-mit-100-privatsphaere> [Zugriff am: 2025-07-28]
- [27] Sicherheit bei der HomeKit-kommunikation. [Online]. Verfügbar: <https://support.apple.com/de-de/guide/security/sec3a881ccb1/web> [Zugriff am: 2025-07-28]
- [28] Die aqara-markengeschichte. [Online]. Verfügbar: <https://www.aqara.com/de/about-us/brand-story/> [Zugriff am: 2025-07-28]
- [29] Die besten zigbee-smart-home-geräte. [Online]. Verfügbar: <https://eu.aqara.com/de-eu/blogs/neuigkeiten/die-besten-zigbee-smart-home-geraete> [Zugriff am: 2025-07-28]
- [30] Aqara hub – aqara. [Online]. Verfügbar: <https://www.aqara.com/en/product/smart-home-hub/> [Zugriff am: 2025-07-28]
- [31] "IST Sensor Feuchte- und Temperatur-Sensor 1 St. HYT 271 Messbereich: 0 - 100 % rF (L x B x H) 10.2 x 5.1 x 1.8 mm kaufen." [Online]. Verfügbar: <https://www.conrad.de/de/p/ist-sensor-feuchte-und-temperatur-sensor-1-st-hyt-271-messbereich-0-100-rf-l-x-b-x-h-10-2-x-5-1-x-1-8-mm-505675.html> [Zugriff am: 2025-01-25]
- [32] "CC2D23-SIP | Amphenol Digital Temperatursensor und Feuchtigkeitssensor ±2% THT, 4-Pin, I2C -40 bis 125 °C. | RS." [Online]. Verfügbar: <https://de.rs-online.com/web/p/temperatur-und-luftfeuchtigkeit-sensoren/2105091> [Zugriff am: 2025-01-28]
- [33] "Joy-it SEN-CCS811V1 Sensor-Modul 1 St. kaufen." [Online]. Verfügbar: <https://www.conrad.de/de/p/joy-it-sen-ccs811v1-sensor-modul-1-st-1884871.html> [Zugriff am: 2025-01-25]
- [34] r. e. G. I. T. webmaster@reichelt.de, "OMRON Thermischer MEMS-Präsenzsensoren, 1x1 Element, 5...50 °C | Bewegungs-/Präsenzsensoren günstig kaufen | reichelt elektronik." [Online]. Verfügbar: https://www.reichelt.de/de/de/shop/produkt/thermischer_mems-praesenzsensor_1x1_element_5_50_c-293436 [Zugriff am: 2025-01-25]

- [35] "D6T-44L-06H | Omron D6T Wärmesensor IC-Näherungssensor 3m, Modul 4-Pin | RS." [Online]. Verfügbar: <https://de.rs-online.com/web/p/naherungssensoren-ics/2043349> [Zugriff am: 2025-01-28]
- [36] R. Sathiyamoorthy, "I2c-bus," Klasse E4p, Embedded Control, Hr.Felser, HTI Burgdorf. [Online]. Verfügbar: https://www.mikrocontroller.net/attachment/372171/I2C_bus.pdf [Zugriff am: 2025-01-25]
- [37] "I2C-Bus - einfach und verständlich erklärt!" [Online]. Verfügbar: <http://fmh-studios.de/theorie/informationstechnik/i2c-bus/> [Zugriff am: 2025-01-28]
- [38] "Application note humidity modules hyt." [Online]. Verfügbar: https://www.ist-ag.com/sites/default/files/downloads/AHHYTM_E.pdf [Zugriff am: 2025-01-25]
- [39] "Ccs811_datasheet-ds000459.pdf." [Online]. Verfügbar: https://cdn.sparkfun.com/assets/learn_tutorials/1/4/3/CCS811_Datasheet-DS000459.pdf [Zugriff am: 2025-01-25]
- [40] "Mems thermal sensors d6t user's manual." [Online]. Verfügbar: https://cdn-reichelt.de/documents/datenblatt/B400/D6T_MANUAL-ENPDF.pdf [Zugriff am: 2025-01-25]
- [41] "Mems thermal sensors d6t." [Online]. Verfügbar: https://cdn-reichelt.de/documents/datenblatt/B400/D6T_DB-EN.pdf [Zugriff am: 2025-01-25]
- [42] "MQTT - The Standard for IoT Messaging." [Online]. Verfügbar: <https://mqtt.org/> [Zugriff am: 2025-03-03]
- [43] C. de Schryver, "Vorlesung netzwerk- und bustechnik (nubt): T09 metagateways onlyoff-ice," 2024.
- [44] "MQTT QoS: Understanding Quality of Service." [Online]. Verfügbar: <https://assetwolf.com/learn/mqtt-qos-understanding-quality-of-service> [Zugriff am: 2025-03-10]
- [45] "Matter-Vorteile #1: Lokale Verbindung." [Online]. Verfügbar: <https://matter-smarthome.de/vorteile/matter-vorteile-1-die-lokale-verbindung/> [Zugriff am: 2025-03-04]
- [46] "Matter-Vorteile #4: Sicherheit und Verschlüsselung." [Online]. Verfügbar: <https://matter-smarthome.de/vorteile/matter-vorteile-4-sicherheit-verschluesselung/> [Zugriff am: 2025-03-10]

- [47] "Matter-Vorteile #5: Multi-Admin-Betrieb." [Online]. Verfügbar: <https://matter-smarthome.de/vorteile/matter-vorteile-5-multi-admin/> [Zugriff am: 2025-03-05]
- [48] F.-O. Grün, "Was ist ein Matter Fabric?" May 2023. [Online]. Verfügbar: <https://matter-smarthome.de/wissen/was-ist-ein-matter-fabric/> [Zugriff am: 2025-03-11]
- [49] —, "Was ist ein Matter Controller?" Jan. 2024. [Online]. Verfügbar: <https://matter-smarthome.de/wissen/was-ist-ein-matter-controller/> [Zugriff am: 2025-03-04]
- [50] —, "Was ist ein Matter Binding?" Sep. 2023. [Online]. Verfügbar: <https://matter-smarthome.de/wissen/was-ist-ein-binding-in-matter/> [Zugriff am: 2025-03-11]
- [51] —, "Was ist Matter Casting?" Jul. 2024. [Online]. Verfügbar: <https://matter-smarthome.de/wissen/was-ist-matter-casting/> [Zugriff am: 2025-03-11]
- [52] —, "Was ist eine Matter Bridge?" Jun. 2023. [Online]. Verfügbar: <https://matter-smarthome.de/wissen/was-ist-eine-matter-bridge/> [Zugriff am: 2025-03-04]
- [53] "Matter-Vorteile #2: das Funkprotokoll Thread." [Online]. Verfügbar: <https://matter-smarthome.de/vorteile/matter-vorteile-2-das-funkprotokoll-thread/> [Zugriff am: 2025-03-11]
- [54] F.-O. Grün, "Was ist ein Thread Border Router?" Jul. 2023. [Online]. Verfügbar: <https://matter-smarthome.de/wissen/was-ist-ein-thread-border-router/> [Zugriff am: 2025-03-11]
- [55] "esp32-c6-devkitc-1-pin-layout.png (1600×1120)." [Online]. Verfügbar: https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32c6/_images/esp32-c6-devkitc-1-pin-layout.png [Zugriff am: 2025-01-25]
- [56] "vscode-esp-idf-extension/README.md at master · espressif/vscode-esp-idf-extension." [Online]. Verfügbar: <https://github.com/espressif/vscode-esp-idf-extension/blob/master/README.md> [Zugriff am: 2025-01-25]
- [57] "Using WSL in Windows - - - ESP-IDF Extension for VSCode latest documentation." [Online]. Verfügbar: <https://docs.espressif.com/projects/vscode-esp-idf-extension/en/latest/additionalfeatures/wsl.html> [Zugriff am: 2025-04-05]
- [58] "Standard toolchain setup for linux and macOS - ESP32-c6 - - ESP-IDF programming guide latest documentation." [Online]. Verfügbar: <https://docs.espressif.com/projects/>

- esp-idf/en/latest/esp32c6/get-started/linux-macos-setup.html [Zugriff am: 2025-04-09]
- [59] "2. Developing with the SDK - ESP32-C6 - – Espressif's SDK for Matter latest documentation." [Online]. Verfügbar: <https://docs.espressif.com/projects/esp-matter/en/latest/esp32c6/developing.html#developing-your-product> [Zugriff am: 2025-04-04]
- [60] F. v. Dorsselaer, "dorssel/usbipd-win," original-date: 2020-10-18T21:44:11Z. [Online]. Verfügbar: <https://github.com/dorssel/usbipd-win> [Zugriff am: 2025-06-21]
- [61] "esp-matter/examples/sensors at main · espressif/esp-matter." [Online]. Verfügbar: <https://github.com/espressif/esp-matter/tree/main/examples/sensors> [Zugriff am: 2025-06-20]
- [62] "Downloads – oracle VirtualBox." [Online]. Verfügbar: <https://www.virtualbox.org/wiki/Downloads> [Zugriff am: 2025-06-01]
- [63] H. Assistant, "Windows." [Online]. Verfügbar: <https://www.home-assistant.io/installation/windows/> [Zugriff am: 2025-05-30]
- [64] DennisM00, "DennisM00/sensors," original-date: 2025-04-09T15:59:05Z. [Online]. Verfügbar: <https://github.com/DennisM00/sensors> [Zugriff am: 2025-07-22]

Abkürzungsverzeichnis

Abkürzung	Bedeutung
ggf.	gegebenenfalls
z.B.	zum Beispiel
d.h.	das heißt
I2C	inter-integrated-circuit-bus
SDA	shift data-Leitung
SCL	shift clock-Leitung
GND	ground
R/W	read oder write
ACK	acknowledge
VOC	volatile organic compounds
TVOC	totale volatile organic compounds
CO ₂	carbon dioxide
eCO ₂	equivalent carbon dioxide
ppm	parts per million
ppb	parts per billion
PTAT	proportional to absolut temperature
PEC	packet error check code
MQTT	message queueing telemetry transport
Qos	quality of service
DAC	device attestation certificate
PASE	password authenticated session establishment
ACL	access control list
IDE	integrated development environment
TLS	transport layer security
FFD	fully functional device
RFD	reduced functional device
PCB	printed circuit board

Glossar

Begriff	Beschreibung
Breadboard-Reihe	Das Breadboard ist in mehrere Teile aufgeteilt. Dabei ist der größte Teil in ein Raster von Spalten und Reihen aufgeteilt, wobei die Spalten von a-j und die Reihen von 1-63 gegliedert sind.
Thermopile-Sensor	Ein Gerät, das an ein Thermoelement kaskadiert wird, um die Spannung zu erhöhen. Thermoelemente werden so angeordnet, dass die heißen Verbindungen nebeneinander liegen.[40]
Scatternet	Ein Scatternet ist eine Gruppe von maximal 10 kleinerer Netze (Piconetze). Diese sind über Bluetooth miteinander verbunden und können auch über größere Entfernung miteinander Daten austauschen.
AES-128	Das AES-128 ist ein Verschlüsselungssystem. Dabei steht AES für Advanced Encryption Standard und die Zahl 128 für die Anzahl von Bits, welche für den Schlüssel verwendet wurde.

Tabellenverzeichnis

2.1	Smart-Home Systems	7
3.1	Multisensor (Temperatur- und Feuchtigkeitssensor)	8
3.2	Luftqualitätssensor	8
3.3	Standardpräsenzmelder	8

Abbildungsverzeichnis

3.1	IST HYT 271	10
3.2	aktiver Sensor des HYT271	10
3.3	Aufbau des Sensors	10
3.4	HYT271 Measurement Request Protokoll	11
3.5	HYT271 Data Fetch Protokoll	11
3.6	HYT271 Umrechnung in reale Werte	12
3.7	Joy-it SEN-CCS811V1	12
3.8	CCS811 Protokoll	13
3.9	CCS811 Aufbau des Algorithm Results Registers	13
3.10	Omron D6T1A01	13
3.11	D6T1A01 Protokoll	14
3.12	D6T1A01 Signalverlauf	15
3.13	MQTT Modell	16
4.1	Hardwareaufbau	20
4.2	Sensors-Struktur	21
4.3	I2C Struktur der Funktionen	22
4.4	Pin Header	24
4.5	PCB Layout	25
4.6	Multisensorgerät in HomeAssistant	26
4.7	Projektstruktur Kommunikation	27
4.8	Einsatzbeispiel	28
6.1	PCB Verbesserung (EasyEDA)	30

Anhang

A.1 WSL: flash + monitor

```
dennismakein@DESKTOP-R97H8J5:~$ get_idf
Checking "python3" ...
Python 3.10.12
"python3" has been detected
Activating ESP-IDF 5.5
Setting IDF_PATH to '/home/dennismakein/esp/esp-idf'.
* Checking python version ... 3.10.12
* Checking python dependencies ... OK
* Deactivating the current ESP-IDF environment (if any) ... OK
* Establishing a new ESP-IDF environment ... OK
* Identifying shell ... bash
* Detecting outdated tools in system ... OK - no outdated tools found
* Shell completion ... Autocompletion code generated
```

Done! You can now compile ESP-IDF projects.
Go to the project directory and run:

```
idf.py build
dennismakein@DESKTOP-R97H8J5:~$ get_matter
Setting ESP_MATTER_PATH to '/home/dennismakein/esp/esp-matter'
dennismakein@DESKTOP-R97H8J5:~$ set_cache
dennismakein@DESKTOP-R97H8J5:~$ cd esp/sensors
dennismakein@DESKTOP-R97H8J5:~/esp/sensors$ idf.py flash monitor
Executing action: flash
Serial port /dev/ttyACM0
Connecting....
Detecting chip type... ESP32-C6
Running ninja in directory /home/dennismakein/esp/sensors/build
Executing "ninja flash"...
[0/1] Re-running CMake...
-- Component directory /home/dennismakein/esp/esp-matter/examples/common/blemesh_platform does
not contain a CMakeLists.txt file. No component will be added
-- Component directory /home/dennismakein/esp/esp-matter/examples/common/cmake_common does
not contain a CMakeLists.txt file. No component will be added
-- Component directory /home/dennismakein/esp/esp-matter/examples/common/external_platform does
not contain a CMakeLists.txt file. No component will be added
-- Component directory /home/dennismakein/esp/esp-matter/examples/common/relinker does not contain
a CMakeLists.txt file. No component will be added
-- Component directory /home/dennismakein/esp/esp-matter/examples/common/utls does not contain a
CMakeLists.txt file. No component will be added
-- Minimal build - OFF
-- ccache will be used for faster recompilation
-- Building ESP-IDF components for target esp32c6
NOTICE: Skipping optional dependency: espressif/cmake_utilities
NOTICE: Skipping optional dependency: espressif/esp_wifi_remote
NOTICE: Skipping optional dependency: espressif/cmake_utilities
NOTICE: Skipping optional dependency: espressif/esp_wifi_remote
NOTICE: Skipping optional dependency: espressif/cmake_utilities
NOTICE: Skipping optional dependency: espressif/esp_wifi_remote
NOTICE: Skipping optional dependency: espressif/cmake_utilities
NOTICE: Skipping optional dependency: espressif/esp_wifi_remote
NOTICE:
Following dependencies have new versions available:
Dependency "espressif/cbor": "0.6.0~1" -> "0.6.0~2"
Dependency "espressif/esp_delta_ota": "1.1.1" -> "1.1.2"
```

Dependency "espressif/esp_diagnostics": "1.2.1" -> "1.2.3"
Dependency "espressif/esp_insights": "1.2.4" -> "1.2.7"
Dependency "espressif/esp_lcd_touch_cst816s": "1.0.6" -> "1.1.0"
Dependency "espressif/esp_secure_cert_mgr": "2.5.0" -> "2.5.1"
Dependency "espressif/led_strip": "3.0.1" -> "3.0.1~1"
Dependency "espressif/rmaker_common": "1.4.6" -> "1.4.12"
Dependency "lvgl/lvgl": "9.2.2" -> "9.3.0"
Consider running "idf.py update-dependencies" to update your lock file.
NOTICE: Processing 30 dependencies:
NOTICE: [1/30] espressif/button (3.5.0)
NOTICE: [2/30] espressif/cbor (0.6.0~1)
NOTICE: [3/30] espressif/cmake_utilities (0.5.3)
NOTICE: [4/30] espressif/esp-serial-flasher (0.0.11)
NOTICE: [5/30] espressif/esp_esp8266 (1.2.1)
NOTICE: [6/30] espressif/esp_delta_ota (1.1.1)
NOTICE: [7/30] espressif/esp_diag_data_store (1.0.2)
NOTICE: [8/30] espressif/esp_diagnostics (1.2.1)
NOTICE: [9/30] espressif/esp_encrypted_img (2.3.0)
NOTICE: [10/30] espressif/esp_insights (1.2.4)
NOTICE: [11/30] espressif/esp_lcd_gc9a01 (1.2.0)
NOTICE: [12/30] espressif/esp_lcd_ili9341 (1.2.0)
NOTICE: [13/30] espressif/esp_lcd_touch (1.1.2)
NOTICE: [14/30] espressif/esp_lcd_touch_cst816s (1.0.6)
NOTICE: [15/30] espressif/esp_lcd_touch_ft5x06 (1.0.7)
NOTICE: [16/30] espressif/esp_lcd_touch_gt1151 (1.0.5~2)
NOTICE: [17/30] espressif/esp_lcd_touch_gt911 (1.1.3)
NOTICE: [18/30] espressif/esp_lcd_touch_tt21100 (1.1.1)
NOTICE: [19/30] espressif/esp_lvgl_port (2.6.0)
NOTICE: [20/30] espressif/esp_rcp_update (1.3.1)
NOTICE: [21/30] espressif/esp_secure_cert_mgr (2.5.0)
NOTICE: [22/30] espressif/jsmn (1.1.0)
NOTICE: [23/30] espressif/json_generator (1.1.2)
NOTICE: [24/30] espressif/json_parser (1.0.3)
NOTICE: [25/30] espressif/led_indicator (0.9.3)
NOTICE: [26/30] espressif/led_strip (3.0.1)
NOTICE: [27/30] espressif/mdns (1.8.2)
NOTICE: [28/30] espressif/rmaker_common (1.4.6)
NOTICE: [29/30] lvgl/lvgl (9.2.2)
NOTICE: [30/30] idf (5.5.0)
-- Found Python3: /home/dennismakein/.espressif/python_env/idf5.5_py3.10_env/bin/python3.10 (found version "3.10.12") found components: Interpreter
NOTICE: Skipping optional dependency: espressif/esp_wifi_remote
NOTICE: Skipping optional dependency: espressif/cmake_utilities
-- Project sdkconfig file /home/dennismakein/esp/sensors/sdkconfig
warning: default on the choice symbol SEC_CERT_DAC_PROVIDER (defined at /home/dennismakein/esp/esp-matter/connectedhomeip/connectedhomeip/config/esp32/components/chip/Kconfig:837, /home/dennismakein/esp/esp-matter/components/esp_matter/Kconfig:48) will have no effect, as defaults do not affect choice symbols
warning: the choice symbol SEC_CERT_DAC_PROVIDER (defined at /home/dennismakein/esp/esp-matter/connectedhomeip/connectedhomeip/config/esp32/components/chip/Kconfig:837, /home/dennismakein/esp/esp-matter/components/esp_matter/Kconfig:48) is defined with a prompt outside the choice
info: INFO: Symbol SEC_CERT_DAC_PROVIDER defined in multiple locations (see below). Please check if this is a correct behavior or a random name match:
/home/dennismakein/esp/esp-matter/components/esp_matter/Kconfig:48

/home/dennismakein/esp/esp-matter/connectedhomeip/connectedhomeip/config/esp32/components/chip/
Kconfig:837

info: INFO: Symbol ENV_MAX_LEDS defined in multiple locations (see below). Please check if this is a correct behavior or a random name match:

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:348

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:343

info: INFO: Symbol ENV_GPIO_RANGE_MIN defined in multiple locations (see below). Please check if this is a correct behavior or a random name match:

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:973

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:1008

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:1042

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:919

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:991

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:1025

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:902

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:937

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:955

info: INFO: Symbol ENV_GPIO_RANGE_MAX defined in multiple locations (see below). Please check if this is a correct behavior or a random name match:

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:941

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:1029

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:1046

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:977

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:906

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:995

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:923

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:1012

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:959

info: INFO: Symbol ENV_GPIO_IN_RANGE_MAX defined in multiple locations (see below). Please check if this is a correct behavior or a random name match:

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:999

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:928

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:982

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:1016

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:1033

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:946

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:964

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:1050

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:910

info: INFO: Symbol ENV_GPIO_OUT_RANGE_MAX defined in multiple locations (see below). Please check if this is a correct behavior or a random name match:

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:1037

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:968

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:950

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:932

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:1054

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:914

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:986

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:1020

/home/dennismakein/esp/sensors/managed_components/espressif__esp_esp_esp_generic/Kconfig:1003

Loading defaults file /home/dennismakein/esp/sensors/sdkconfig.defaults...

warning: unknown kconfig symbol 'ENABLE_WIFI_AP' assigned to 'n' in

/home/dennismakein/esp/sensors/sdkconfig.defaults

-- Adding linker script /home/dennismakein/esp/esp-idf/components/riscv/ld/rom.api.ld

```

-- USING O3
-- App "sensors" version: 1.0
-- Adding linker script /home/dennismakein/esp/sensors/build/esp-idf/esp_system/ld/memory.ld
-- Adding linker script /home/dennismakein/esp/sensors/build/esp-idf/esp_system/ld/sections.ld.in
-- Adding linker script /home/dennismakein/esp/esp-idf/components/esp_rom/esp32c6/ld/esp32c6.rom.ld
-- Adding linker script
/home/dennismakein/esp/esp-idf/components/esp_rom/esp32c6/ld/esp32c6.rom.api.ld
-- Adding linker script
/home/dennismakein/esp/esp-idf/components/esp_rom/esp32c6/ld/esp32c6.rom.rvfp.ld
-- Adding linker script
/home/dennismakein/esp/esp-idf/components/esp_rom/esp32c6/ld/esp32c6.rom.wdt.ld
-- Adding linker script
/home/dennismakein/esp/esp-idf/components/esp_rom/esp32c6/ld/esp32c6.rom.systimer.ld
-- Adding linker script
/home/dennismakein/esp/esp-idf/components/esp_rom/esp32c6/ld/esp32c6.rom.version.ld
-- Adding linker script
/home/dennismakein/esp/esp-idf/components/esp_rom/esp32c6/ld/esp32c6.rom.phy.ld
-- Adding linker script
/home/dennismakein/esp/esp-idf/components/esp_rom/esp32c6/ld/esp32c6.rom.coexist.ld
-- Adding linker script
/home/dennismakein/esp/esp-idf/components/esp_rom/esp32c6/ld/esp32c6.rom.net80211.ld
-- Adding linker script
/home/dennismakein/esp/esp-idf/components/esp_rom/esp32c6/ld/esp32c6.rom.pp.ld
-- Adding linker script
/home/dennismakein/esp/esp-idf/components/esp_rom/esp32c6/ld/esp32c6.rom.libc.ld
-- Adding linker script
/home/dennismakein/esp/esp-idf/components/esp_rom/esp32c6/ld/esp32c6.rom.libc-suboptimal_for_misa
ligned_mem.ld
-- Adding linker script
/home/dennismakein/esp/esp-idf/components/esp_rom/esp32c6/ld/esp32c6.rom.newlib.ld
-- Adding linker script
/home/dennismakein/esp/esp-idf/components/esp_rom/esp32c6/ld/esp32c6.rom.newlib-normal.ld
-- Adding linker script
/home/dennismakein/esp/esp-idf/components/esp_rom/esp32c6/ld/esp32c6.rom.heap.ld
-- Adding linker script
/home/dennismakein/esp/esp-idf/components/soc/esp32c6/ld/esp32c6.peripherals.ld
-- BUTTON: 3.5.0
-- LVGL version: 9.2.2
-- LED_INDICATOR: 0.9.3
-- ESP_LCD_GC9A01: 1.2.0
-- ESP_LCD_ILI9341: 1.2.0
ESP Insights Project commit: b0c2938
-- Components: app_bridge app_reset app_trace app_update bootloader bootloader_support bt chip
cmock console cxx driver efuse esp-tls esp_adc esp_app_format esp_bootloader_format esp_coex
esp_common esp_driver_ana_cmpr esp_driver_bitscrambler esp_driver_cam esp_driver_dac
esp_driver_gpio esp_driver_gptimer esp_driver_i2c esp_driver_i2s esp_driver_isp esp_driver_jpeg
esp_driver_ledc esp_driver_mcpwm esp_driver_parlio esp_driver_pcnt esp_driver_ppa esp_driver_rmt
esp_driver_sdio esp_driver_sdm esp_driver_sdmmc esp_driver_sdspi esp_driver_spi
esp_driver_touch_sens esp_driver_tsens esp_driver_twai esp_driver_uart esp_driver_usb_serial_jtag
esp_eth esp_event esp_gdbstub esp_hid esp_http_client esp_http_server esp_https_ota
esp_https_server esp_hw_support esp_lcd esp_local_ctrl esp_matter esp_matter_bridge
esp_matter_console esp_matter_controller esp_matter_ota_provider esp_matter_rainmaker esp_mm
esp_netif esp_netif_stack esp_partition esp_phy esp_pm esp_psram esp_ringbuf esp_rom esp_security
esp_system esp_tee esp_timer esp_vfs_console esp_wifi espcoredump espressif_button
espressif_cbor espressif_cmake_utilities espressif_esp-serial-flasher espressif_esp_bsp_generic

```

espressif__esp_delta_ota espressif__esp_diag_data_store espressif__esp_diagnostics
espressif__esp_encrypted_img espressif__esp_insights espressif__esp_lcd_gc9a01
espressif__esp_lcd_ili9341 espressif__esp_lcd_touch espressif__esp_lcd_touch_cst816s
espressif__esp_lcd_touch_ft5x06 espressif__esp_lcd_touch_gt1151 espressif__esp_lcd_touch_gt911
espressif__esp_lcd_touch_tt21100 espressif__esp_lvgl_port espressif__esp_rcp_update
espressif__esp_secure_cert_mgr espressif__jsmn espressif__json_generator espressif__json_parser
espressif__led_indicator espressif__led_strip espressif__mdns espressif__rmaker_common esptool_py
fatfs freertos hal heap http_parser idf_test ieee802154 json log lvgl__lvgl lwip main mbedtls mqtt newlib
nvs_flash nvs_sec_provider openthread partition_table protobuf-c protocomm pthread riscv rt sdmmc soc
spi_flash spiiffs tcp_transport ulp unity usb vfs wear_levelling wifi_provisioning wpa_supplicant
-- Component paths: /home/dennismakein/esp/esp-matter/examples/common/app_bridge
/home/dennismakein/esp/esp-matter/examples/common/app_reset
/home/dennismakein/esp/esp-idf/components/app_trace
/home/dennismakein/esp/esp-idf/components/app_update
/home/dennismakein/esp/esp-idf/components/bootloader
/home/dennismakein/esp/esp-idf/components/bootloader_support
/home/dennismakein/esp/esp-idf/components/bt
/home/dennismakein/esp/esp-matter/connectedhomeip/config/esp32/components/chip
/home/dennismakein/esp/esp-idf/components/cmock
/home/dennismakein/esp/esp-idf/components/console /home/dennismakein/esp/esp-idf/components/cxx
/home/dennismakein/esp/esp-idf/components/driver /home/dennismakein/esp/esp-idf/components/efuse
/home/dennismakein/esp/esp-idf/components/esp-tls
/home/dennismakein/esp/esp-idf/components/esp_adc
/home/dennismakein/esp/esp-idf/components/esp_app_format
/home/dennismakein/esp/esp-idf/components/esp_bootloader_format
/home/dennismakein/esp/esp-idf/components/esp_coex
/home/dennismakein/esp/esp-idf/components/esp_common
/home/dennismakein/esp/esp-idf/components/esp_driver_ana_cmpr
/home/dennismakein/esp/esp-idf/components/esp_driver_bitscrambler
/home/dennismakein/esp/esp-idf/components/esp_driver_cam
/home/dennismakein/esp/esp-idf/components/esp_driver_dac
/home/dennismakein/esp/esp-idf/components/esp_driver_gpio
/home/dennismakein/esp/esp-idf/components/esp_driver_gptimer
/home/dennismakein/esp/esp-idf/components/esp_driver_i2c
/home/dennismakein/esp/esp-idf/components/esp_driver_i2s
/home/dennismakein/esp/esp-idf/components/esp_driver_isp
/home/dennismakein/esp/esp-idf/components/esp_driver_jpeg
/home/dennismakein/esp/esp-idf/components/esp_driver_ledc
/home/dennismakein/esp/esp-idf/components/esp_driver_mcpwm
/home/dennismakein/esp/esp-idf/components/esp_driver_parlio
/home/dennismakein/esp/esp-idf/components/esp_driver_pcnt
/home/dennismakein/esp/esp-idf/components/esp_driver_ppa
/home/dennismakein/esp/esp-idf/components/esp_driver_rmt
/home/dennismakein/esp/esp-idf/components/esp_driver_sdio
/home/dennismakein/esp/esp-idf/components/esp_driver_sdm
/home/dennismakein/esp/esp-idf/components/esp_driver_sdmmc
/home/dennismakein/esp/esp-idf/components/esp_driver_sdspi
/home/dennismakein/esp/esp-idf/components/esp_driver_spi
/home/dennismakein/esp/esp-idf/components/esp_driver_touch_sens
/home/dennismakein/esp/esp-idf/components/esp_driver_tsens
/home/dennismakein/esp/esp-idf/components/esp_driver_twai
/home/dennismakein/esp/esp-idf/components/esp_driver_uart
/home/dennismakein/esp/esp-idf/components/esp_driver_usb_serial_jtag
/home/dennismakein/esp/esp-idf/components/esp_eth
/home/dennismakein/esp/esp-idf/components/esp_event

/home/dennismakein/esp/esp-idf/components/esp_gdbstub
/home/dennismakein/esp/esp-idf/components/esp_hid
/home/dennismakein/esp/esp-idf/components/esp_http_client
/home/dennismakein/esp/esp-idf/components/esp_http_server
/home/dennismakein/esp/esp-idf/components/esp_https_ota
/home/dennismakein/esp/esp-idf/components/esp_https_server
/home/dennismakein/esp/esp-idf/components/esp_hw_support
/home/dennismakein/esp/esp-idf/components/esp_lcd
/home/dennismakein/esp/esp-idf/components/esp_local_ctrl
/home/dennismakein/esp/esp-matter/components/esp_matter
/home/dennismakein/esp/esp-matter/components/esp_matter_bridge
/home/dennismakein/esp/esp-matter/components/esp_matter_console
/home/dennismakein/esp/esp-matter/components/esp_matter_controller
/home/dennismakein/esp/esp-matter/components/esp_matter_ota_provider
/home/dennismakein/esp/esp-matter/components/esp_matter_rainmaker
/home/dennismakein/esp/esp-idf/components/esp_mm
/home/dennismakein/esp/esp-idf/components/esp_netif
/home/dennismakein/esp/esp-idf/components/esp_netif_stack
/home/dennismakein/esp/esp-idf/components/esp_partition
/home/dennismakein/esp/esp-idf/components/esp_phy
/home/dennismakein/esp/esp-idf/components/esp_pm
/home/dennismakein/esp/esp-idf/components/esp_psram
/home/dennismakein/esp/esp-idf/components/esp_ringbuf
/home/dennismakein/esp/esp-idf/components/esp_rom
/home/dennismakein/esp/esp-idf/components/esp_security
/home/dennismakein/esp/esp-idf/components/esp_system
/home/dennismakein/esp/esp-idf/components/esp_tee
/home/dennismakein/esp/esp-idf/components/esp_timer
/home/dennismakein/esp/esp-idf/components/esp_vfs_console
/home/dennismakein/esp/esp-idf/components/esp_wifi
/home/dennismakein/esp/esp-idf/components/espcoredump
/home/dennismakein/esp/sensors/managed_components/espressif_button
/home/dennismakein/esp/sensors/managed_components/espressif_cbor
/home/dennismakein/esp/sensors/managed_components/espressif_cmake_utilities
/home/dennismakein/esp/sensors/managed_components/espressif_esp-serial-flasher
/home/dennismakein/esp/sensors/managed_components/espressif_esp_esp-bsp-generic
/home/dennismakein/esp/sensors/managed_components/espressif_esp_delta_ota
/home/dennismakein/esp/sensors/managed_components/espressif_esp_diag_data_store
/home/dennismakein/esp/sensors/managed_components/espressif_esp_diagnostics
/home/dennismakein/esp/sensors/managed_components/espressif_esp_encrypted_img
/home/dennismakein/esp/sensors/managed_components/espressif_esp_insights
/home/dennismakein/esp/sensors/managed_components/espressif_esp_lcd_gc9a01
/home/dennismakein/esp/sensors/managed_components/espressif_esp_lcd_ili9341
/home/dennismakein/esp/sensors/managed_components/espressif_esp_lcd_touch
/home/dennismakein/esp/sensors/managed_components/espressif_esp_lcd_touch_cst816s
/home/dennismakein/esp/sensors/managed_components/espressif_esp_lcd_touch_ft5x06
/home/dennismakein/esp/sensors/managed_components/espressif_esp_lcd_touch_gt1151
/home/dennismakein/esp/sensors/managed_components/espressif_esp_lcd_touch_gt911
/home/dennismakein/esp/sensors/managed_components/espressif_esp_lcd_touch_tt21100
/home/dennismakein/esp/sensors/managed_components/espressif_esp_lvgl_port
/home/dennismakein/esp/sensors/managed_components/espressif_esp_rcp_update
/home/dennismakein/esp/sensors/managed_components/espressif_esp_secure_cert_mgr
/home/dennismakein/esp/sensors/managed_components/espressif_jsmn
/home/dennismakein/esp/sensors/managed_components/espressif_json_generator
/home/dennismakein/esp/sensors/managed_components/espressif_json_parser

```

/home/dennismakein/esp/sensors/managed_components/espressif__led_indicator
/home/dennismakein/esp/sensors/managed_components/espressif__led_strip
/home/dennismakein/esp/sensors/managed_components/espressif__mdns
/home/dennismakein/esp/sensors/managed_components/espressif__rmaker_common
/home/dennismakein/esp/esp-idf/components/esptool_py
/home/dennismakein/esp/esp-idf/components/fatfs /home/dennismakein/esp/esp-idf/components/freertos
/home/dennismakein/esp/esp-idf/components/hal /home/dennismakein/esp/esp-idf/components/heap
/home/dennismakein/esp/esp-idf/components/http_parser
/home/dennismakein/esp/esp-idf/components/idf_test
/home/dennismakein/esp/esp-idf/components/ieee802154
/home/dennismakein/esp/esp-idf/components/json /home/dennismakein/esp/esp-idf/components/log
/home/dennismakein/esp/sensors/managed_components/lvgl__lvgl
/home/dennismakein/esp/esp-idf/components/lwip /home/dennismakein/esp/sensors/main
/home/dennismakein/esp/esp-idf/components/mbedtls /home/dennismakein/esp/esp-idf/components/mqtt
/home/dennismakein/esp/esp-idf/components/newlib
/home/dennismakein/esp/esp-idf/components/nvs_flash
/home/dennismakein/esp/esp-idf/components/nvs_sec_provider
/home/dennismakein/esp/esp-idf/components/openthread
/home/dennismakein/esp/esp-idf/components/partition_table
/home/dennismakein/esp/esp-idf/components/protobuf-c
/home/dennismakein/esp/esp-idf/components/protocomm
/home/dennismakein/esp/esp-idf/components/pthread /home/dennismakein/esp/esp-idf/components/riscv
/home/dennismakein/esp/esp-idf/components/rt /home/dennismakein/esp/esp-idf/components/sdmmc
/home/dennismakein/esp/esp-idf/components/soc /home/dennismakein/esp/esp-idf/components/spi_flash
/home/dennismakein/esp/esp-idf/components/spiffs
/home/dennismakein/esp/esp-idf/components/tcp_transport
/home/dennismakein/esp/esp-idf/components/ulp /home/dennismakein/esp/esp-idf/components/unity
/home/dennismakein/esp/esp-idf/components/usb /home/dennismakein/esp/esp-idf/components/vfs
/home/dennismakein/esp/esp-idf/components/wear_levelling
/home/dennismakein/esp/esp-idf/components/wifi_provisioning
/home/dennismakein/esp/esp-idf/components/wpa_supplicant
-- Configuring done
-- Generating done
-- Build files have been written to: /home/dennismakein/esp/sensors/build
[1/1] cd /home/dennismakein/esp/sensors/build/bootloader/esp-idf/esptool_py &&
/home/dennismakein/.espressif/python_e...on_table/check_sizes.py --offset 0xc000 bootloader 0x0
/home/dennismakein/esp/sensors/build/bootloader/bootloader.bi
Bootloader binary size 0x57d0 bytes. 0x6830 bytes (54%) free.
[0/1] Regenerating ninja files
ninja: no work to do.
[4/5] cd /home/dennismakein/esp/sensors/build/esp-idf/esptool_py &&
/home/dennismakein/.espressif/python_env/idf5.5_p...e/dennismakein/esp/sensors/build/partition_table/p
artition-table.bin /home/dennismakein/esp/sensors/build/sensors.bi
sensors.bin binary size 0x19c200 bytes. Smallest app partition is 0x1e0000 bytes. 0x43e00 bytes (14%)
free.
[4/5] cd /home/dennismakein/esp/esp-idf/components/esptool_py && /usr/bin/cmake -D
IDF_PATH=/home/dennismakein/esp/es...RY=/home/dennismakein/esp/sensors/build -P
/home/dennismakein/esp/esp-idf/components/esptool_py/run_serial_tool.cmak
esptool.py --chip esp32c6 -p /dev/ttyACM0 -b 460800 --before=default_reset --after=hard_reset
write_flash --flash_mode dio --flash_freq 80m --flash_size 8MB 0x0 bootloader/bootloader.bin 0x20000
sensors.bin 0xc000 partition_table/partition-table.bin 0x1d000 ota_data_initial.bin
esptool.py v4.9.dev8
Serial port /dev/ttyACM0
Connecting...
Chip is ESP32-C6 (QFN40) (revision v0.1)

```

Features: WiFi 6, BT 5, IEEE802.15.4
Crystal is 40MHz
USB mode: USB-Serial/JTAG
MAC: f0:f5:bd:ff:fe:01:8d:94
BASE MAC: f0:f5:bd:01:8d:94
MAC_EXT: ff:fe
Uploading stub...
Running stub...
Stub running...
Changing baud rate to 460800
Changed.
Configuring flash size...
Flash will be erased from 0x00000000 to 0x00005fff...
Flash will be erased from 0x00020000 to 0x001bcfff...
Flash will be erased from 0x0000c000 to 0x0000cfff...
Flash will be erased from 0x0001d000 to 0x0001efff...
SHA digest in image updated
Compressed 22480 bytes to 13907...
Writing at 0x00000000... (100 %)
Wrote 22480 bytes (13907 compressed) at 0x00000000 in 0.3 seconds (effective 543.6 kbit/s)...
Hash of data verified.
Compressed 1688064 bytes to 1025318...
Writing at 0x001b66ae... (100 %)
Wrote 1688064 bytes (1025318 compressed) at 0x00020000 in 8.7 seconds (effective 1553.3 kbit/s)...
Hash of data verified.
Compressed 3072 bytes to 181...
Writing at 0x0000c000... (100 %)
Wrote 3072 bytes (181 compressed) at 0x0000c000 in 0.0 seconds (effective 631.9 kbit/s)...
Hash of data verified.
Compressed 8192 bytes to 31...
Writing at 0x0001d000... (100 %)
Wrote 8192 bytes (31 compressed) at 0x0001d000 in 0.1 seconds (effective 986.8 kbit/s)...
Hash of data verified.

Leaving...
Hard resetting via RTS pin...
Executing action: monitor
Running idf_monitor in directory /home/dennismakein/esp/sensors
Executing "/home/dennismakein/.espressif/python_env/idf5.5_py3.10_env/bin/python
/home/dennismakein/esp/esp-idf/tools/idf_monitor.py -p /dev/ttyACM0 -b 115200 --toolchain-prefix
riscv32-esp-elf- --target esp32c6 --revision 0 --decode-panic backtrace
/home/dennismakein/esp/sensors/build/sensors.elf
/home/dennismakein/esp/sensors/build/bootloader/bootloader.elf -m
'/home/dennismakein/.espressif/python_env/idf5.5_py3.10_env/bin/python'
'/home/dennismakein/esp/esp-idf/tools/idf.py'..."
--- esp-idf-monitor 1.6.2 on /dev/ttyACM0 115200
--- Quit: Ctrl+] | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H
ESP-ROM:esp32c6-20220919
Build:Sep 19 2022
rst:0x15 (USB_UART_HPSYS),boot:0xc (SPI_FAST_FLASH_BOOT)
Saved PC:0x4080a530
--- 0x4080a530: rv_utils_wait_for_intr at
/home/dennismakein/esp/esp-idf/components/riscv/include/riscv/rv_utils.h:82
--- (inlined by) esp_cpu_wait_for_intr at
/home/dennismakein/esp/esp-idf/components/esp_hw_support/cpu.c:62

```

SPIWP:0xee
mode:DIO, clock div:2
load:0x40875730,len:0x1734
load:0x4086b910,len:0xea8
--- 0x4086b910: esp_bootloader_get_description at
/home/dennismakein/esp/esp-idf/components/esp_bootloader_format/esp_bootloader_desc.c:39
(discriminator 1)

load:0x4086e610,len:0x31a0
--- 0x4086e610: esp_flash_encryption_enabled at
/home/dennismakein/esp/esp-idf/components/bootloader_support/src/flash_encrypt.c:89

entry 0x4086b91a
--- 0x4086b91a: call_start_cpu0 at
/home/dennismakein/esp/esp-idf/components/bootloader/subproject/main/bootloader_start.c:25

I (23) boot: ESP-IDF v5.5-dev-2916-g1c468f6825 2nd stage bootloader
I (23) boot: compile time May 23 2025 18:39:56
I (24) boot: chip revision: v0.1
I (24) boot: efuse block revision: v0.2
I (28) boot.esp32c6: SPI Speed      : 80MHz
I (32) boot.esp32c6: SPI Mode      : DIO
I (36) boot.esp32c6: SPI Flash Size : 8MB
I (39) boot: Enabling RNG early entropy source...
I (44) boot: Partition Table:
I (46) boot: ## Label            Usage            Type ST Offset   Length
I (53) boot: 0 esp_secure_cert   unknown          3f 06 0000d000 00002000
I (59) boot: 1 nvs                WiFi data       01 02 00010000 0000c000
I (66) boot: 2 nvs_keys           NVS keys        01 04 0001c000 00001000
I (72) boot: 3 otadata            OTA data        01 00 0001d000 00002000
I (79) boot: 4 phy_init            RF data         01 01 0001f000 00001000
I (85) boot: 5 ota_0               OTA app         00 10 00020000 001e0000
I (92) boot: 6 ota_1             OTA app         00 11 00200000 001e0000
I (98) boot: 7 fctry             WiFi data       01 02 003e0000 00006000
I (105) boot: End of partition table
I (109) esp_image: segment 0: paddr=00020020 vaddr=42140020 size=3a400h (238592) map
I (161) esp_image: segment 1: paddr=0005a428 vaddr=40800000 size=05bf0h ( 23536) load
I (167) esp_image: segment 2: paddr=00060020 vaddr=42000020 size=13e918h (1304856) map
I (412) esp_image: segment 3: paddr=0019e940 vaddr=40805bf0 size=194b8h (103608) load
I (435) esp_image: segment 4: paddr=001b7e00 vaddr=4081f0b0 size=043ach ( 17324) load
I (440) esp_image: segment 5: paddr=001bc1b4 vaddr=50000000 size=0001ch (   28) load
I (448) boot: Loaded app from partition at offset 0x20000
I (448) boot: Disabling RNG early entropy source...
I (461) cpu_start: Unicore app
I (469) cpu_start: Pro cpu start user code
I (469) cpu_start: cpu freq: 160000000 Hz
I (470) app_init: Application information:
I (470) app_init: Project name:   sensors
I (473) app_init: App version:    1.0
I (477) app_init: Compile time:   Jul 12 2025 15:16:36
I (482) app_init: ELF file SHA256: dbfed4ece...
I (486) app_init: ESP-IDF:        v5.5-dev-2916-g1c468f6825
I (492) efuse_init: Min chip rev:  v0.0
I (496) efuse_init: Max chip rev:  v0.99

```

I (499) efuse_init: Chip rev: v0.1
I (503) heap_init: Initializing. RAM available for dynamic allocation:
I (510) heap_init: At 40837BB0 len 00044A60 (274 KiB): RAM
I (515) heap_init: At 4087C610 len 00002F54 (11 KiB): RAM
I (520) heap_init: At 5000001C len 00003FCC (15 KiB): RTCRAM
I (526) spi_flash: detected chip: generic
I (529) spi_flash: flash io: dio
W (532) i2c: This driver is an old driver, please migrate your application code to adapt 'driver/i2c_master.h'
I (543) sleep_gpio: Configure to isolate all GPIO pins in sleep state
I (548) sleep_gpio: Enable automatic switching of GPIO sleep configuration
I (555) coexist: coex firmware version: 88dafd1
I (559) coexist: coexist rom version 5b8dcfa
I (563) main_task: Started on CPU0
I (563) main_task: Calling app_main()
I (593) button: IoT Button Version: 3.5.0
E (593) esp_matter_cluster: Config is NULL or mandatory feature_flags are missing.
E (593) i2c: i2c_driver_delete(481): i2c driver install error
I (793) i2c: ccs811 initialized successfully
I (793) i2c: hyt271 initialized successfully
I (793) i2c: d6t1a01 initialized successfully
I (793) pp: pp rom version: 5b8dcfa
I (793) net80211: net80211 rom version: 5b8dcfa
I (803) wifi:wifi driver task: 40842544, prio:23, stack:6656, core=0
I (813) wifi:wifi firmware version: a017748
I (813) wifi:wifi certification version: v7.0
I (813) wifi:config NVS flash: enabled
I (813) wifi:config nano formatting: disabled
I (823) wifi:mac_version:HAL_MAC_ESP32AX_761,ut_version:N, band mode:0x1
I (823) wifi:Init data frame dynamic rx buffer num: 32
I (833) wifi:Init static rx mgmt buffer num: 5
I (833) wifi:Init management short buffer num: 32
I (843) wifi:Init dynamic tx buffer num: 32
I (843) wifi:Init static tx FG buffer num: 2
I (843) wifi:Init static rx buffer size: 1700 (rxctrl:92, csi:512)
I (853) wifi:Init static rx buffer num: 10
I (853) wifi:Init dynamic rx buffer num: 32
I (863) wifi_init: rx ba win: 6
I (863) wifi_init: accept mbox: 6
I (863) wifi_init: tcpip mbox: 32
I (873) wifi_init: udp mbox: 6
I (873) wifi_init: tcp mbox: 6
I (873) wifi_init: tcp tx win: 5760
I (873) wifi_init: tcp rx win: 5760
I (883) wifi_init: tcp mss: 1440
I (883) wifi_init: WiFi IRAM OP enabled
I (883) wifi_init: WiFi RX IRAM OP enabled
I (893) wifi_init: WiFi SLP IRAM OP enabled
I (913) chip[DL]: NVS set: chip-counters/reboot-count = 16 (0x10)
I (913) chip[DL]: Real time clock set to 946684800 (2000-01-01 00:00:00 UTC)
I (913) BLE_INIT: Using main XTAL as clock source
I (913) BLE_INIT: ble controller commit:[ec6437f]
I (923) BLE_INIT: Bluetooth MAC: f0:f5:bd:01:8d:96
I (923) phy_init: phy_version 331,5b89037,Mar 3 2025,16:01:12
I (983) phy: libbtbb version: ec2ecba, Mar 3 2025, 16:01:27

I (983) NimBLE: GAP procedure initiated: stop advertising.

I (993) CHIP[DL]: BLE host-controller synced

I (1493) chip[DL]: Starting ESP WiFi layer

W (1493) wifi:ACK_TAB0 :0x 90a0b, QAM16:0x9 (24Mbps), QPSK:0xa (12Mbps), BPSK:0xb (6Mbps)

W (1493) wifi:CTS_TAB0 :0x 90a0b, QAM16:0x9 (24Mbps), QPSK:0xa (12Mbps), BPSK:0xb (6Mbps)

W (1503) wifi:(agc)0x600a7128:0xd216fa2e, min.avgNF:0xce->0xd2(dB), RCalCount:0x16f, min.RRssi:0xa2e(-93.12)

W (1503) wifi:(TB)WDEV_PWR_TB_MCS0:19

W (1513) wifi:(TB)WDEV_PWR_TB_MCS1:19

W (1513) wifi:(TB)WDEV_PWR_TB_MCS2:19

W (1513) wifi:(TB)WDEV_PWR_TB_MCS3:19

W (1523) wifi:(TB)WDEV_PWR_TB_MCS4:19

W (1523) wifi:(TB)WDEV_PWR_TB_MCS5:19

W (1523) wifi:(TB)WDEV_PWR_TB_MCS6:18

W (1533) wifi:(TB)WDEV_PWR_TB_MCS7:18

W (1533) wifi:(TB)WDEV_PWR_TB_MCS8:17

W (1533) wifi:(TB)WDEV_PWR_TB_MCS9:15

W (1543) wifi:(TB)WDEV_PWR_TB_MCS10:15

W (1543) wifi:(TB)WDEV_PWR_TB_MCS11:15

I (1553) wifi:11ax coex: WDEVAX_PTI0(0x55777555), WDEVAX_PTI1(0x00003377).

I (1553) wifi:mode : sta (f0:f5:bd:01:8d:94)

I (1563) wifi:enable tsf

I (1563) chip[DL]: Posting ESPSystemEvent: Wifi Event with eventId : 43

W (1563) wifi:Haven't to connect to a suitable AP now!

I (1573) chip[DL]: Attempting to connect WiFi station interface

I (1583) chip[DL]: WiFi station state change: NotConnected -> Connecting

I (1583) wifi:new:<2,0>, old:<1,0>, ap:<255,255>, sta:<2,0>, prof:1, snd_ch_cfg:0x0

I (1593) wifi:(connect)dot11_authmode:0x3, pairwise_cipher:0x3, group_cipher:0x1

I (1603) wifi:state: init -> auth (0xb0)

I (1603) chip[DL]: Posting ESPSystemEvent: Wifi Event with eventId : 2

I (1613) chip[DL]: Posting ESPSystemEvent: Wifi Event with eventId : 43

I (1603) chip[DL]: Done driving station state, nothing else to do...

I (1623) wifi:state: auth -> assoc (0x0)

I (1623) chip[SVR]: Initializing subscription resumption storage...

I (1633) chip[SVR]: Server initializing...

I (1633) chip[TS]: Last Known Good Time: 2025-05-24T15:26:32

I (1643) chip[FP]: Fabric index 0x1 was retrieved from storage. Compressed FabricId 0xCE59028EE8FB1476, FabricId 0x00000000215F69DF, NodeId 0x00000000C48BF455, VendorId 0x1384

I (1653) chip[FP]: Fabric index 0x2 was retrieved from storage. Compressed FabricId 0x1C0344479B0D92E2, FabricId 0x0000000000000002, NodeId 0x0000000000000001, VendorId 0x134B

I (1673) chip[DMG]: AccessControl: initializing

I (1673) chip[DMG]: Examples::AccessControlDelegate::Init

I (1683) chip[DMG]: AccessControl: setting

I (1683) chip[DMG]: DefaultAclStorage: initializing

I (1693) chip[DMG]: DefaultAclStorage: 2 entries loaded

I (1693) chip[SVR]: WARNING: mTestEventTriggerDelegate is null

I (1643) wifi:state: assoc -> run (0x10)

I (1723) chip[ZCL]: Using ZAP configuration...

I (1723) esp_matter_cluster: Cluster plugin init common callback

I (1723) chip[DMG]: AccessControlCluster: initializing

I (1733) chip[ZCL]: WRITE ERR: ep 0 clus 0x0000_0030 attr 0x0000_0000 not supported

```

I (1733) chip[ZCL]: Initiating Admin Commissioning cluster.
I (1743) chip[SVR]: Fabric already commissioned. Disabling BLE advertisement
I (1733) wifi:(trc)phytype:CBW20-SGI, snr:33, maxRate:144, highestRateIdx:0
W (1753) wifi:(trc)band:2G, phymode:3, highestRateIdx:0, lowestRateIdx:11, dataSchedTableSize:14
I (1763) wifi:(trc)band:2G, rate(L-MCS6, rateIdx:2), ampdu(rate:L-MCS6, schedIdx(2, stop:8)), snr:33,
ampduState:wait operational
I (1773) wifi:ifidx:0, rssi:-61, nf:-94, phytype(0x3, CBW20-SGI), phymode(0x3, 11bgn), max_rate:144,
he:0, vht:0, ht:1
I (1783) wifi:(ht)max.RxAMPDULenExponent:3(65535 bytes), MMSS:5(4 us)
I (1763) chip[IN]: CASE Server enabling CASE session setups
I (1803) chip[SVR]: Joining Multicast groups
I (1803) chip[SVR]: Server Listening...
I (1803) wifi:connected with FRITZ!Box 7530 SL-TP:2.4GHz, aid = 6, channel 2, BW20, bssid =
1c:3b:f3:52:d0:00
I (1813) wifi:security: WPA2-PSK, phy:11bgn, rssi:-61, cipher(pairwise:0x3, group:0x1), pmf:0,
I (1823) wifi:pm start, type: 1, twt_start:0

I (1823) wifi:pm start, type:1, aid:0x6, trans-BSSID:1c:3b:f3:52:d0:00, BSSID[5]:0x0,
mbssid(max-indicator:0, index:0), he:0
I (1833) wifi:dp: 1, bi: 102400, li: 3, scale listen interval from 307200 us to 307200 us
I (1843) wifi:set rx beacon pti, rx_bcn_pti: 10, bcn_timeout: 25000, mt_pti: 10, mt_time: 10000
I (1853) wifi:[ADDBA]TX addba request, tid:0, dialogtoken:1, bufsize:64, A-MSDU:0(not supported),
policy:1(IMR), ssn:0(0x0)
I (1863) wifi:[ADDBA]TX addba request, tid:7, dialogtoken:2, bufsize:64, A-MSDU:0(not supported),
policy:1(IMR), ssn:0(0x20)
I (1873) wifi:[ADDBA]TX addba request, tid:5, dialogtoken:3, bufsize:64, A-MSDU:0(not supported),
policy:1(IMR), ssn:0(0x0)
I (1813) esp_matter_core: Dynamic endpoint 0 added
I (1893) esp_matter_attribute: ***** R : Endpoint 0x0001's Cluster 0x00000003's Attribute
0x00000001 is 2 *****
I (1903) esp_matter_core: Dynamic endpoint 1 added
I (1903) esp_matter_attribute: ***** R : Endpoint 0x0002's Cluster 0x00000003's Attribute
0x00000001 is 2 *****
I (1913) esp_matter_core: Dynamic endpoint 2 added
I (1923) esp_matter_attribute: ***** R : Endpoint 0x0003's Cluster 0x00000003's Attribute
0x00000001 is 2 *****
I (1933) esp_matter_core: Dynamic endpoint 3 added
I (1933) esp_matter_attribute: ***** R : Endpoint 0x0004's Cluster 0x00000003's Attribute
0x00000001 is 2 *****
I (1943) esp_matter_core: Dynamic endpoint 4 added
I (1953) esp_matter_attribute: ***** R : Endpoint 0x0005's Cluster 0x00000003's Attribute
0x00000001 is 2 *****
I (1963) esp_matter_attribute: ***** R : Endpoint 0x0005's Cluster 0x00000406's Attribute
0x00000001 is 0 *****
I (1973) esp_matter_attribute: ***** R : Endpoint 0x0005's Cluster 0x00000406's Attribute
0x00000002 is 1 *****
I (1983) esp_matter_core: Dynamic endpoint 5 added
I (1883) wifi:AP's beacon interval = 102400 us, DTIM period = 1
I (1993) wifi:[ADDBA]RX addba response, status:0, tid:0/tb:1(0xa1), bufsize:64, batimeout:0, txa_wnd:64
I (2003) chip[DL]: Posting ESPSystemEvent: Wifi Event with eventId : 4
I (2013) main_task: Returned from app_main()
I (2013) chip[DL]: WIFI_EVENT_STA_START
I (2013) chip[DL]: Done driving station state, nothing else to do...
I (2013) wifi:[ADDBA]RX addba response, status:0, tid:7/tb:1(0xa1), bufsize:64, batimeout:0, txa_wnd:64
I (2033) chip[DL]: WIFI_EVENT_STA_CONNECTED

```

I (2033) chip[DL]: WiFi station state change: Connecting -> Connecting_Succeeded
 I (2033) wifi:[ADDBA]RX addba response, status:0, tid:5/tb:1(0xa1), bufsize:64, batimeout:0, txa_wnd:64
 I (2053) chip[DL]: WiFi station state change: Connecting_Succeeded -> Connected
 I (2063) chip[DL]: WiFi station interface connected
 I (2063) chip[DL]: Done driving station state, nothing else to do...
 I (2073) NimBLE: GAP procedure initiated: stop advertising.

 I (2073) chip[DL]: Updating advertising data
 I (3073) esp_netif_handlers: sta ip: 192.168.178.66, mask: 255.255.255.0, gw: 192.168.178.1
 I (3073) chip[DL]: Posting ESPSystemEvent: IP Event with eventId : 0
 I (3073) chip[DL]: IP_EVENT_STA_GOT_IP
 I (3073) chip[DL]: IPv4 address changed on WiFi station interface: 192.168.178.66/255.255.255.0 gateway 192.168.178.1
 I (3083) chip[DL]: IPv4 Internet connectivity ESTABLISHED
 I (3093) chip[DIS]: Updating services using commissioning mode 0
 I (3093) chip[DIS]: CHIP minimal mDNS started advertising.
 I (3103) chip[DIS]: Advertise operational node CE59028EE8FB1476-00000000C48BF455
 I (3113) chip[DIS]: CHIP minimal mDNS configured as 'Operational device'; instance name: CE59028EE8FB1476-00000000C48BF455.
 I (3123) chip[DIS]: mDNS service published: _matter._tcp
 I (3123) chip[DIS]: Advertise operational node 1C0344479B0D92E2-0000000000000001
 I (3133) chip[DIS]: CHIP minimal mDNS configured as 'Operational device'; instance name: 1C0344479B0D92E2-0000000000000001.
 I (3143) chip[DIS]: mDNS service published: _matter._tcp
 I (3793) chip[DL]: Posting ESPSystemEvent: IP Event with eventId : 3
 I (3793) chip[DL]: IP_EVENT_GOT_IP6
 I (3793) chip[DL]: IPv6 addr available. Ready on WIFI_STA_DEF interface: fe80:0000:0000:0000:f2f5:bdff:fe01:8d94
 I (3803) chip[DIS]: Updating services using commissioning mode 0
 I (3803) chip[DIS]: CHIP minimal mDNS started advertising.
 I (3813) chip[DIS]: Advertise operational node CE59028EE8FB1476-00000000C48BF455
 I (3813) chip[DIS]: CHIP minimal mDNS configured as 'Operational device'; instance name: CE59028EE8FB1476-00000000C48BF455.
 W (3823) wifi:<ba-add>idx:0, ifx:0, tid:0, TAHI:0x1000d0, TALO:0x52f33b1c, (ssn:7, win:64, cur_ssn:7), CONF:0xc0000005
 I (3833) chip[DIS]: mDNS service published: _matter._tcp
 I (3843) chip[DIS]: Advertise operational node 1C0344479B0D92E2-0000000000000001
 I (3853) chip[DIS]: CHIP minimal mDNS configured as 'Operational device'; instance name: 1C0344479B0D92E2-0000000000000001.
 I (3863) chip[DIS]: mDNS service published: _matter._tcp
 I (3863) chip[DIS]: Updating services using commissioning mode 0
 I (3873) chip[DIS]: CHIP minimal mDNS started advertising.
 I (3883) chip[DIS]: Advertise operational node CE59028EE8FB1476-00000000C48BF455
 I (3883) chip[DIS]: CHIP minimal mDNS configured as 'Operational device'; instance name: CE59028EE8FB1476-00000000C48BF455.
 I (3893) chip[DIS]: mDNS service published: _matter._tcp
 I (3903) chip[DIS]: Advertise operational node 1C0344479B0D92E2-0000000000000001
 I (3903) chip[DIS]: CHIP minimal mDNS configured as 'Operational device'; instance name: 1C0344479B0D92E2-0000000000000001.
 I (3923) chip[DIS]: mDNS service published: _matter._tcp
 I (3923) chip[SVR]: Server initialization complete
 I (3923) chip[SWU]: Stopping the watchdog timer
 I (3933) chip[SWU]: Starting the periodic query timer, timeout: 86400 seconds
 I (3943) chip[IM]: Resuming 1 subscriptions in 1 seconds
 I (3943) esp_matter_attribute: ***** R : Endpoint 0x0000's Cluster 0x0000002A's Attribute

0x00000002 is 0 *****
I (3953) esp_matter_attribute: ***** R : Endpoint 0x0000's Cluster 0x0000002A's Attribute
0x00000002 is 0 *****
I (3963) esp_matter_attribute: ***** W : Endpoint 0x0000's Cluster 0x0000002A's Attribute
0x00000002 is 1 *****
I (3973) esp_matter_attribute: ***** R : Endpoint 0x0000's Cluster 0x0000002A's Attribute
0x00000003 is null *****
I (4083) chip[DL]: BLE deinit successful and memory reclaimed
I (4083) app_main: BLE deinitialized and memory reclaimed
I (4943) chip[DIS]: Lookup started for 1C0344479B0D92E2-000000000001B669
I (5143) chip[DIS]: Checking node lookup status for 1C0344479B0D92E2-000000000001B669 after 200
ms

CCS811:

Byte 0: 0x01
Byte 1: 0x90
Byte 2: 0x00
Byte 3: 0x00
eCO2: 400 ppm, TVOC: 0 ppb
I (5793) esp_matter_attribute: ***** R : Endpoint 0x0003's Cluster 0x0000040D's Attribute
0x00000000 is null *****
I (5803) esp_matter_attribute: ***** W : Endpoint 0x0003's Cluster 0x0000040D's Attribute
0x00000000 is 400.000000 *****
I (5813) esp_matter_attribute: ***** R : Endpoint 0x0004's Cluster 0x0000042E's Attribute
0x00000000 is null *****
I (5823) esp_matter_attribute: ***** W : Endpoint 0x0004's Cluster 0x0000042E's Attribute
0x00000000 is 0.000000 *****

HYT271:

Byte 0: 00011110
Byte 1: 00100100
Byte 2: 01100011
Byte 3: 11100100
0:30, 3:57
Humiditybytes: 7716, Temperaturebytes: 6393

Humidity: 47.10 %RH, Temperature: 24.39 °C
I (5893) esp_matter_attribute: ***** R : Endpoint 0x0001's Cluster 0x00000402's Attribute
0x00000000 is null *****
I (5903) esp_matter_attribute: ***** W : Endpoint 0x0001's Cluster 0x00000402's Attribute
0x00000000 is 2438 *****

D6T1a01:

PTAT: 24.80 °C, P0: 24.30 °C
Bool: 0
I (5923) esp_matter_attribute: ***** R : Endpoint 0x0002's Cluster 0x00000405's Attribute
0x00000000 is null *****
I (5933) esp_matter_attribute: ***** W : Endpoint 0x0002's Cluster 0x00000405's Attribute
0x00000000 is 4709 *****